

PrimeSim™ Design Insights User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	5
Related Products, Publications, and Trademarks	5
Conventions	5
Customer Support	6
Statement on Inclusivity and Diversity	7

1. Advanced Design Exploration With Design Insights	8
Overview of Design Insights	8
Setting up Design Insights	9
Launching Design Insights	10
Creating diPhysical View	12
Device Bias Voltage Definition File	17
Exporting DSPF by Selected Nets	18
Exploring diPhysical view	19
Analyzing Parasitic	23
Fanout Analysis	29
Understanding Resistance Analysis Results	33
Understanding Elmore Delay Analysis Results	40
Modes of Analysis in Design Insights	43
Histograms for Net Details and Analysis Result	45
Cross Highlighting	47

2. Design Insights Command Reference	52
di::closeResultsCellView	53
di::createResultsCellView	53
di::createParasiticView	54
di::getCapacitors	55
di::getCapAttr	55
di::getCorners	56
di::getElmoreDelayResult	56

Contents

di::getElmoreDelayResultAttr	57
di::getFanoutResult	57
di::getFanoutResultAttr	58
di::getInstAttr	58
di::getInsts	59
di::getLayers	60
di::getModels	60
di::getNetAttr	60
di::getNets	61
di::getNodeAttr	61
di::getResAttr	62
di::getResistanceContributionAttr	63
di::getResistanceResult	63
di::getResistanceResultAttr	64
di::getResistors	64
di::getTermAttr	65
di::getTerms	65
di::getViewType	66
di::listAttrs	66
di::loadResultsCellView	66
di::runElmoreDelayAnalysis	67
di::runFanoutAnalysis	67
di::runResistanceAnalysis	68
Preferences	68

About the User Guide

The *User Guide* describes how to use the Synopsys® tool. The purpose of this manual is to enable users to identify design bottlenecks before running expensive simulations, conduct critical nets probing or analysis on simulation results.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the *PrimeSim Reliability Analysis Release Notes* on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim Design Robustness tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- PrimeWave™ Design Environment
- PrimeSim™ Reliability Analysis

Note:

You must be logged in to SolvNetPlus for the preceding link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Conventions

The following conventions are used in Synopsys documentation.

Contents

Convention	Description
<code>Courier</code>	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Advanced Design Exploration With Design Insights

This chapter introduces the concept of design insights and the Design Insights tool.

- [Overview of Design Insights](#)
- [Setting up Design Insights](#)
- [Launching Design Insights](#)
- [Creating diPhysical View](#)
- [Exporting DSPF by Selected Nets](#)
- [Exploring diPhysical view](#)
- [Analyzing Parasitic](#)
- [Histograms for Net Details and Analysis Result](#)
- [Cross Highlighting](#)

Overview of Design Insights

Parasitic plays a crucial role in custom design. It has a wide impact from delay, crosstalk, device operating point, circuit functionality, and AC characteristics. To get better insight on the design it is important to consider interconnect parasitic and device characteristics together. Design Insights offer deeper design insights for debugging and design optimization for simulation infrastructure and design environment.

The following are the important features of Design Insights:

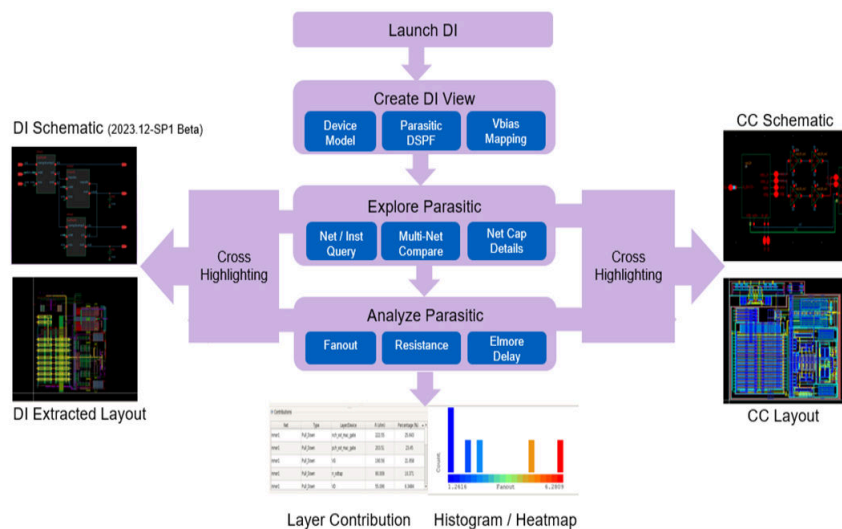
- Perform circuit topology and device aware analyses
- Identify design bottlenecks like fanout and RC contribution before running expensive simulations
- Visualize the post extracted shapes and parasitic with intuitive user interface in the Custom Compiler tool

- Perform interactive and iterative queries to gain better insights on design
- Review multiple variables and its impact on design targets

Design Insights provides a solution to efficiently analyze, visualize parasitics from large DSPF or GPD databases. It provides deep insights into the design and identify bottlenecks in layout before performing expensive simulations. It can also be used along with design and simulation environment to understand simulation failures and perform design exploration.

Design Insights is built on Custom Infrastructure providing intuitive visualizations with cross probing to schematic and layout environments for faster debugging.

Figure 1 Design Insights Flow Diagram



Setting up Design Insights

To run Design Insights with the Custom Compiler tool, the following are the requirements.

License required

- `primewave_re` (UI environment)
- `primesim_ra` or `sim_hsmc` (engine analysis)

Launch flow

1. Add `<your_custominfrastructure_installation_directory>/bin/` to PATH environment variable.
2. Add `<your_custom_compiler_installation_directory>/bin/` to PATH environment variable.
3. Add `<your_primesim_di_installation_directory>/bin/` to PATH environment variable.
4. Run `custom_compiler`.

To run Design Insights with the PrimeWave Design Environment tool, the following are the requirements.

License required

- `primewave_re` (UI environment)
- `primesim_ra` or `sim_hsmc` (engine analysis)

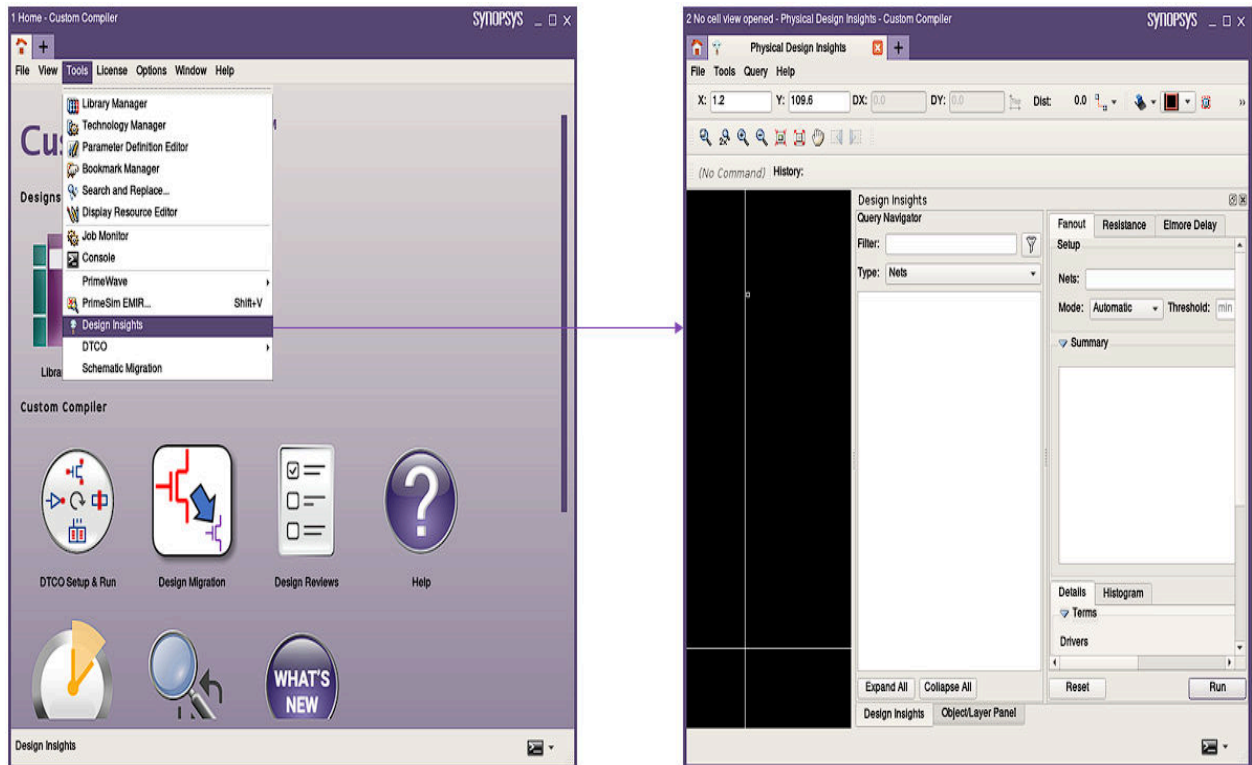
Launch flow

1. Add `<your_custominfrastructure_installation_directory>/bin/` to PATH environment variable.
2. Add `<your_custom_compiler_installation_directory>/bin/` to PATH environment variable.
3. Add `<your_primesim_di_installation_directory>/bin/` to PATH environment variable.
4. Run `primewave_reliability`.

Launching Design Insights

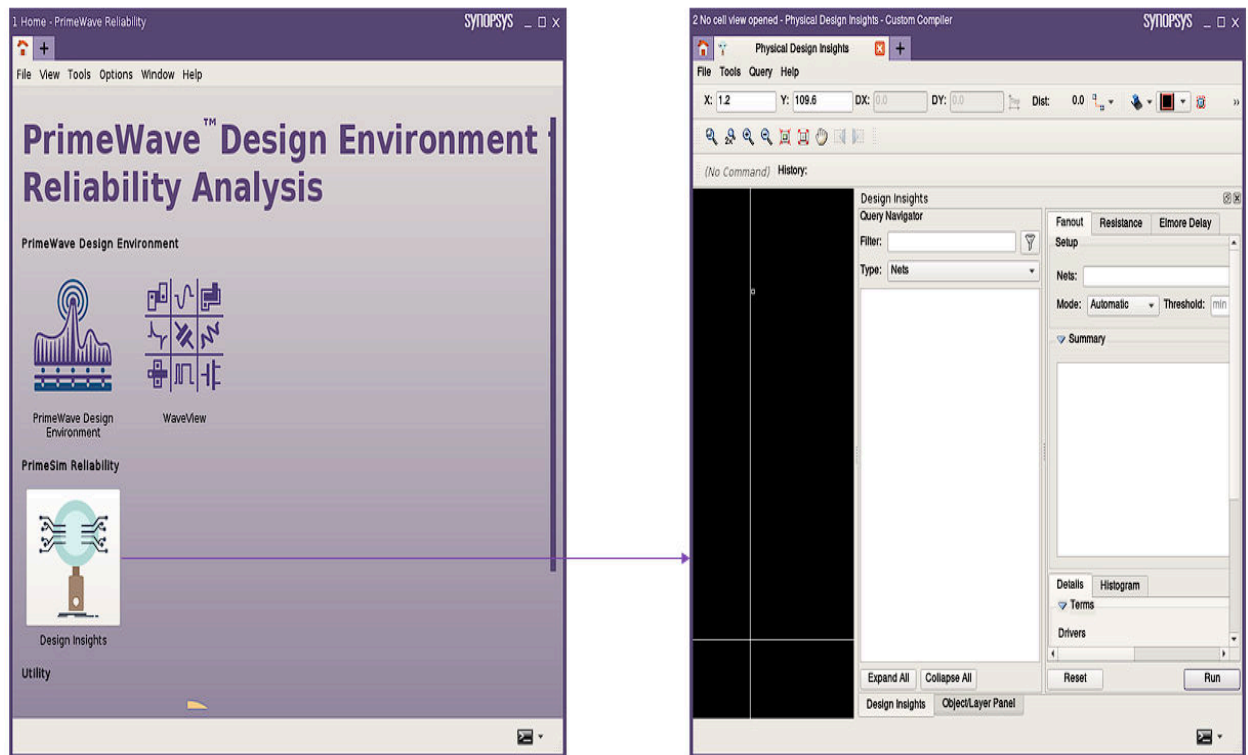
The following figure shows how to launch Design Insights from the **Layout Editor** of the Custom Compiler tool. You can launch Design Insights from Custom Compiler home screen by selecting **Tools > Design Insights**.

Chapter 1: Advanced Design Exploration With Design Insights Launching Design Insights



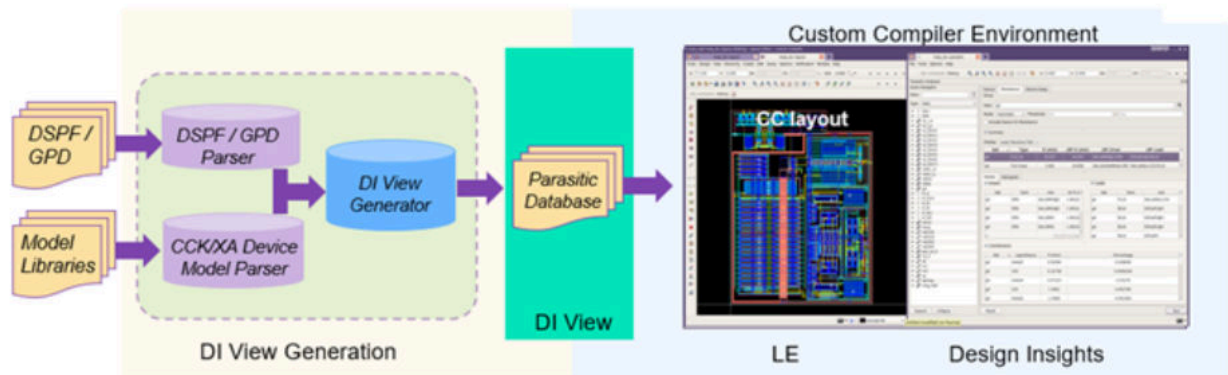
The following figure shows how to launch Design Insights from the PrimeWave Design Environment tool.

Chapter 1: Advanced Design Exploration With Design Insights Creating diPhysical View



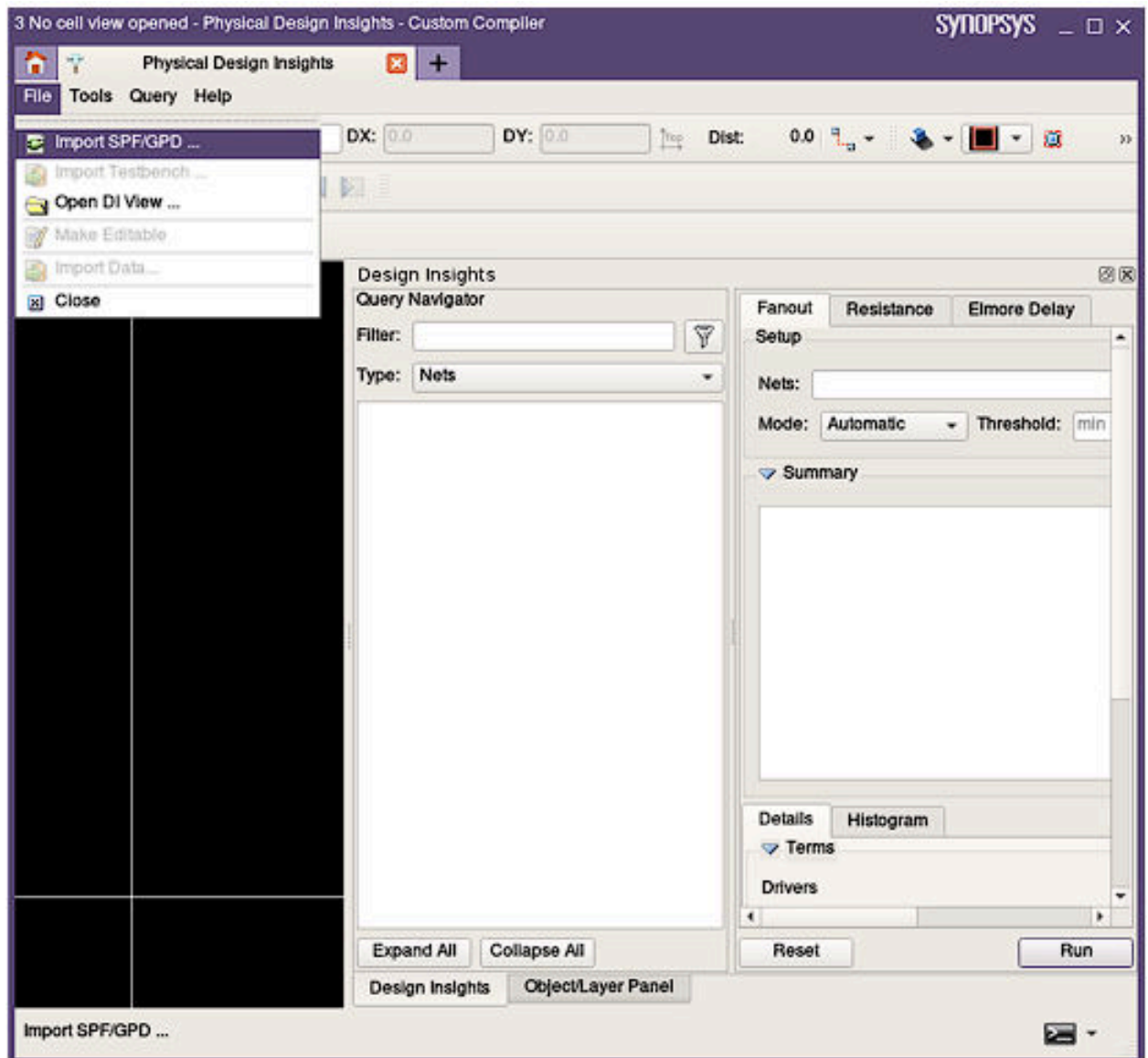
Creating diPhysical View

Design Insights generates design insights view with parasitic data to correlate existing layout.



Complete the following steps to create a design insights view:

1. From the **Design Insights** window, click **File > Import SPF/GPD**.



The **Import SPF/GPD** window appears.

2. In the **Import SPF/GPD** window,
 - **Results Cell View:** Select **Library**, **Cell** and **View** details from the drop-down menu.
 - **Input Data:** Input parasitic types and path.

If you select **DSPF**, it supports flatten DSPF without *include* statement.

- **Vbias Mapping File:** Voltage bias definitions.
- **Default Vbias:** Default voltage bias definitions.

Note:

Either specify a global bias voltage for all devices or set a bias voltage mapping file path.

- **Model Libs:** Specify a model lib path, section, corner (if any) and click to insert a definition item.
- **Skip nets:** Specify names of nets that to be skipped in parasitic view

Ignore Subckt Not Found - Specify if there are any black box cells in the design, enter the cell names in this text box so that the tool can ignore the cells if their subckt definition is not found.

Advanced Options - By default, the SQL and OA database are created within the cell view. You can provide custom paths using the OA Output Root Directory and SQL Output Root Directory widgets.

Import SPF/GPD

Results Cell View

Library:

Cell:

View: diPhysical

Input Data

Type: ☒ DSPF ☐ GPD ☒ Use ITF Layer Name

Path:

Click to add

Vbias Mapping File:

Default Vbias:

Model Libs:

Model Lib	Section	Corner
Click to ...		

Model Syntax:

hspice

Ignore Subckt Not Found:

Nets:

☒ Skip ☐ Select

An empty string mean...

Advanced Options

OA Output Root Directory:

SQL Output Root Directory:

Help

OK

Apply

Close

StarRC Options Required for DSPF Input

The following StarRC options are required for the DSPF input in Design Insights:

StarRC Options	DSPF Information
NETLIST_CONNECT_SECTION: YES	Generates * I and * P in DSPF files
NETLIST_NODE_SECTION: YES	Generates * S in DSPF files
NETLIST_TAIL_COMMENTS: YES	Writes resistors' width, length, and layer information in DSPF files
EXTRA_GEOMETRY_INFO: RES or EXTRA_GEOMETRY_INFO: RES NODE	Writes resistors' bounding box in DSPF files

Device Bias Voltage Definition File

For device evaluations, Design Insights can include device **R_{On}** and terminal cap in analyses to improve accuracy. To do that you must either specify a global bias voltage for all devices or define for specific model or instance in a mapping file. The following is an example of a bias voltage mapping file

```
model      N 1.8
model      P 1.8
model      N25 2.5
model      P25 2.5
instance   ld_l2h|Mpdm<*>|M* 2.5 0.1
```

There are different modes to estimate **R_{On}**. In the following example, **specified** mode means **vgs** and **vds** are specified. For **equivalent**, **min**, or **max** modes, **vdd** is specified and Design Insights estimates **R_{On}** for three predefined scenarios. You can control the mode to be used for a model or instance in bias voltage definition file. The default mode is **equivalent**.

To define mode preference you can also apply the following console command:

```
diPreprocessorRonMode 0|1|2
```

Chapter 1: Advanced Design Exploration With Design Insights Exporting DSPF by Selected Nets

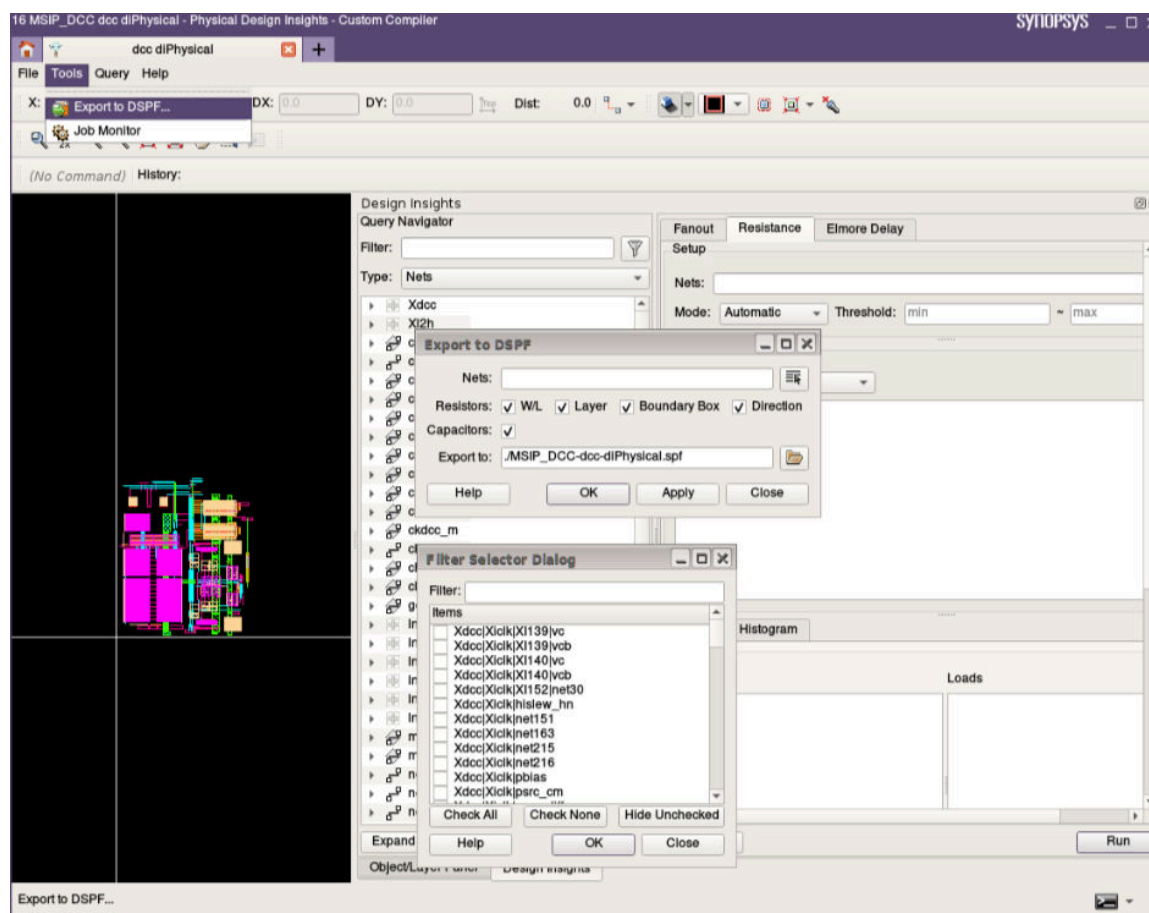
Where,

- 0 (default) : equivalent mode. (average)
- 1 : max mode.
- 2 : min mode.

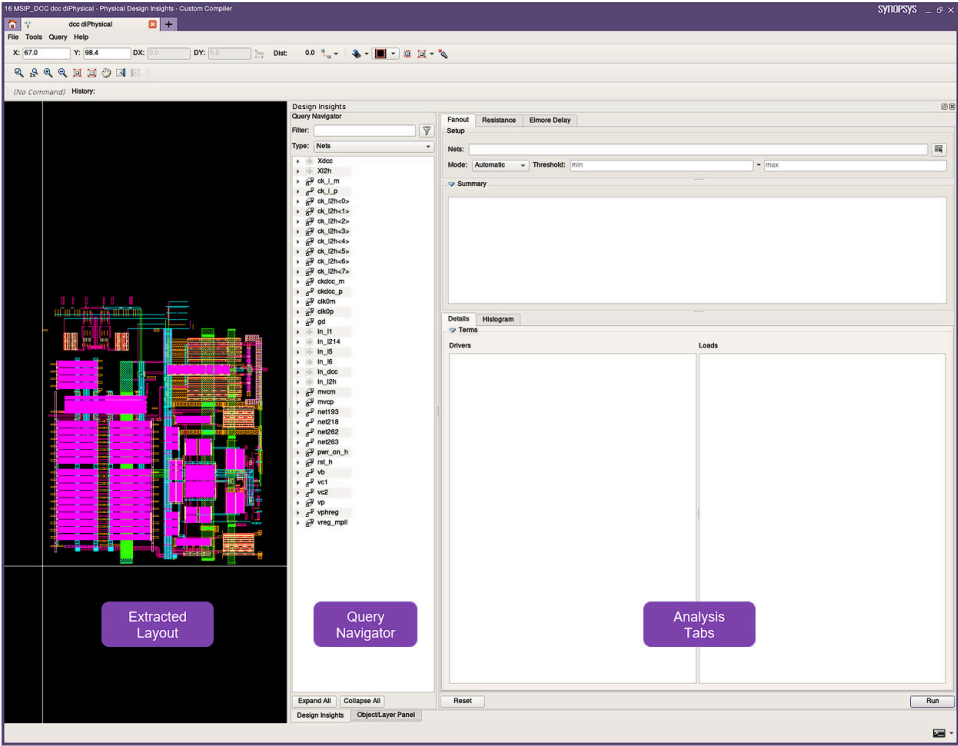
Example: `db::setPrefValue diPreprocessorRonMode -value 0`

Exporting DSPF by Selected Nets

When the parasitic data is established and the Design Insights View is generated you can export a DSPF file with selected nets. To select one or multiple nets, go to **Tools > Export to DSPF** which exports a DSPF file with parasitic R/C only on the selected nets.

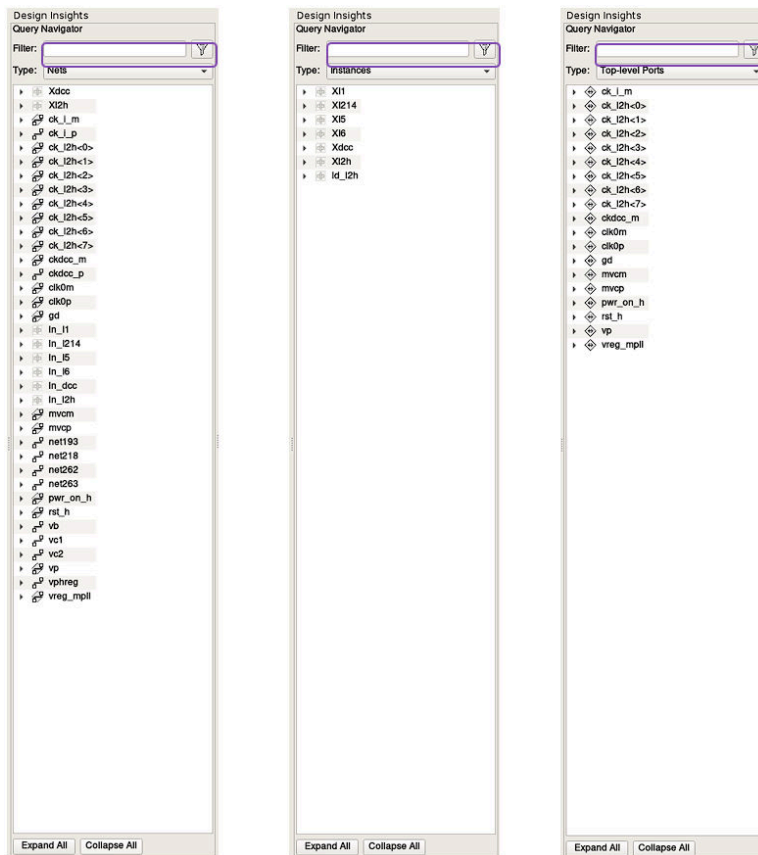


Chapter 1: Advanced Design Exploration With Design Insights
Exploring diPhysical view



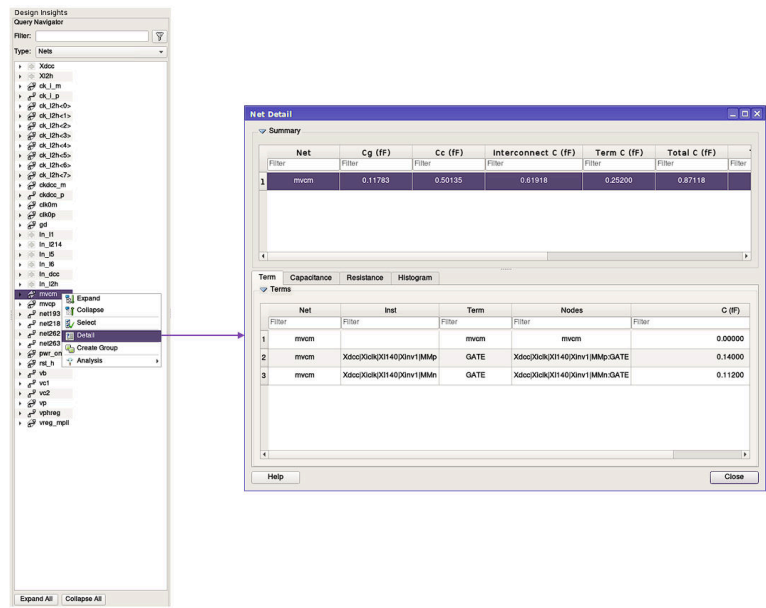
The following procedure shows how to open Query Navigator;

1. Open **Query Navigator** to query **Nets**, **Instance** and **Top-level Ports**.



Chapter 1: Advanced Design Exploration With Design Insights
Exploring diPhysical view

2. Select a specific net and explore parasitic details.



Analyzing Parasitic

Design Insights allows you to run the following analysis:

- Fanout - determines the capability of a device to drive the loads which are connected to it.

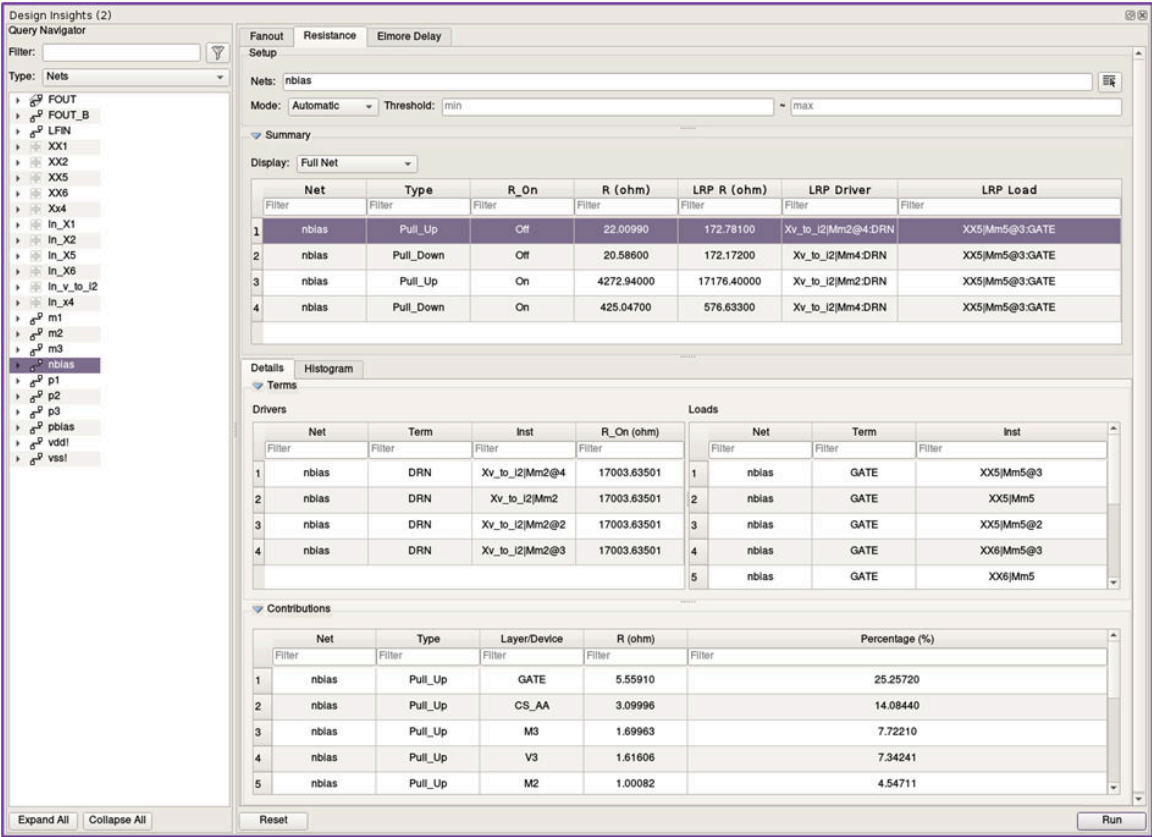
The screenshot shows the 'Design Insights (2)' window with the 'Fanout' tab selected. The 'Setup' section shows 'Nets: nbias pbias', 'Mode: Automatic', and 'Threshold: min'. The 'Summary' table lists the results for four nets.

	Net	Type	Fanout	Driver C (fF)	Load C (fF)	Interconnect C (fF)
1	nbias	Pull_Up	5.33647	21.57720	109.43400	5.71204
2	nbias	Pull_Down	4.19858	27.42500	109.43400	5.71204
3	pbias	Pull_Up	6.13714	21.57720	126.22100	6.20146
4	pbias	Pull_Down	0.31398	421.75200	126.22100	6.20146

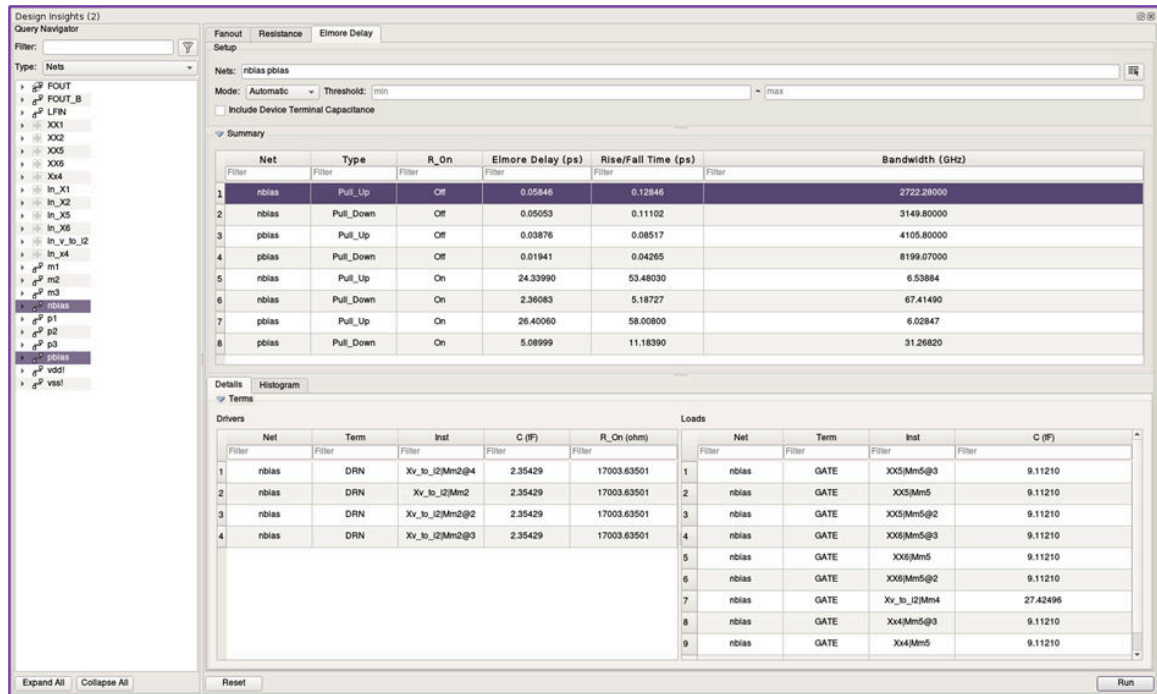
The 'Details' section shows the 'Terms' table, which is divided into 'Drivers' and 'Loads'.

Drivers			Loads				
Net	Term	Inst	Net	Term	Inst		
1	nbias	DRN	Xv_to_i2 Mm2@4	1	nbias	GATE	XX5 Mm5@3
2	nbias	DRN	Xv_to_i2 Mm2	2	nbias	GATE	XX5 Mm5
3	nbias	DRN	Xv_to_i2 Mm2@2	3	nbias	GATE	XX5 Mm5@2
4	nbias	DRN	Xv_to_i2 Mm2@3	4	nbias	GATE	XX6 Mm5@3
				5	nbias	GATE	XX6 Mm5
				6	nbias	GATE	XX6 Mm5@2
				7	nbias	GATE	Xv_to_i2 Mm4
				8	nbias	GATE	Xx4 Mm5@3
				9	nbias	GATE	Xx4 Mm5
				10	nbias	GATE	Xx4 Mm5@2

- Resistance - calculates the effective resistance of all the paths from drivers to loads of the selected nets.



- Elmore Delay - evaluates the delay and check rise or fall time and bandwidth of all the paths from drivers to loads of the selected nets.



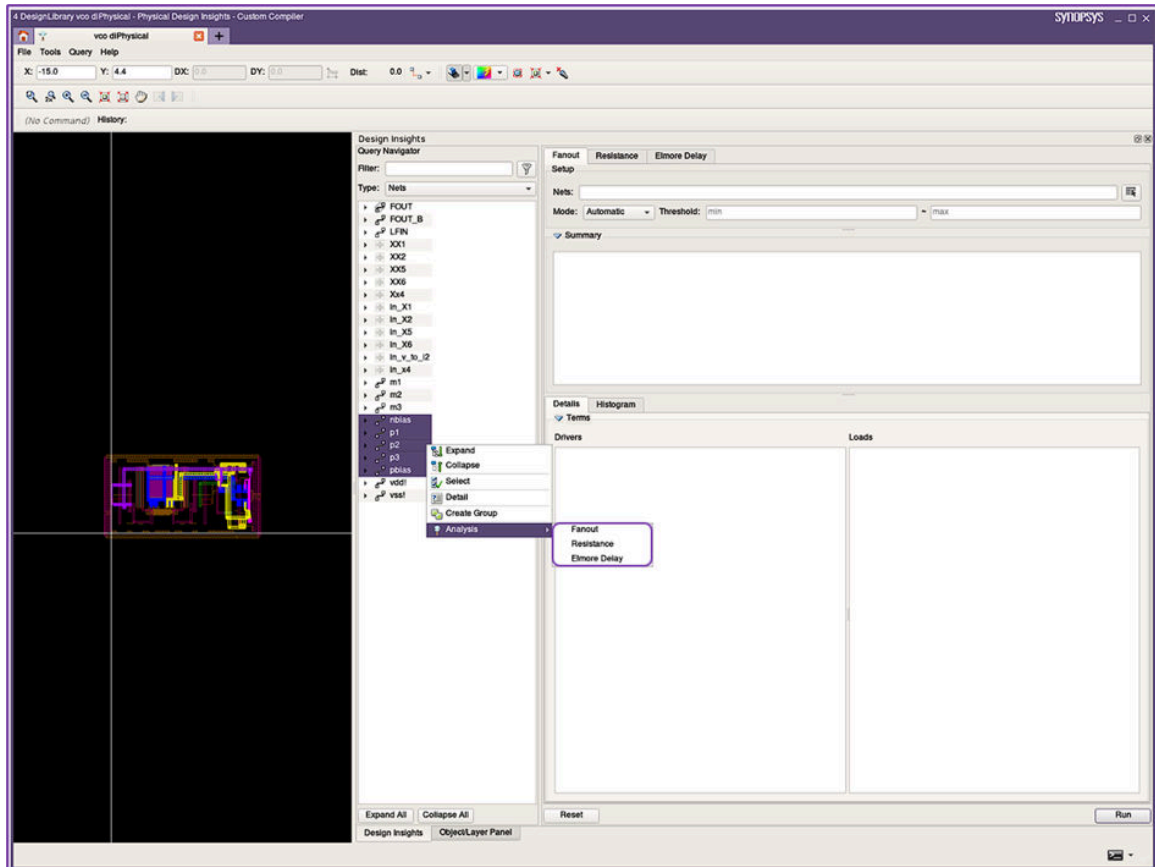
The analysis can be run in the following modes:

- Automatic
- InstPin to Port
- Point-to-Point

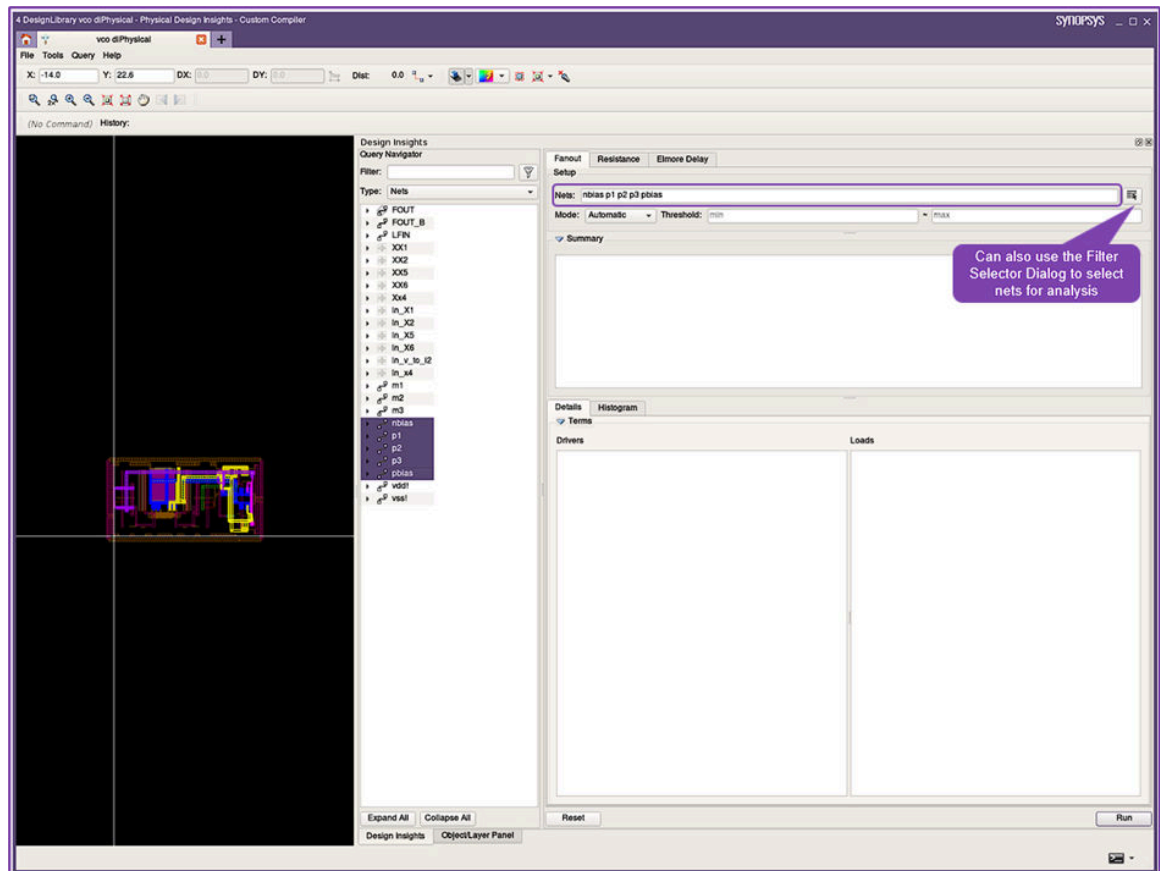
Steps to Run Analysis Using Design Insights

The following steps are common for running any analysis:

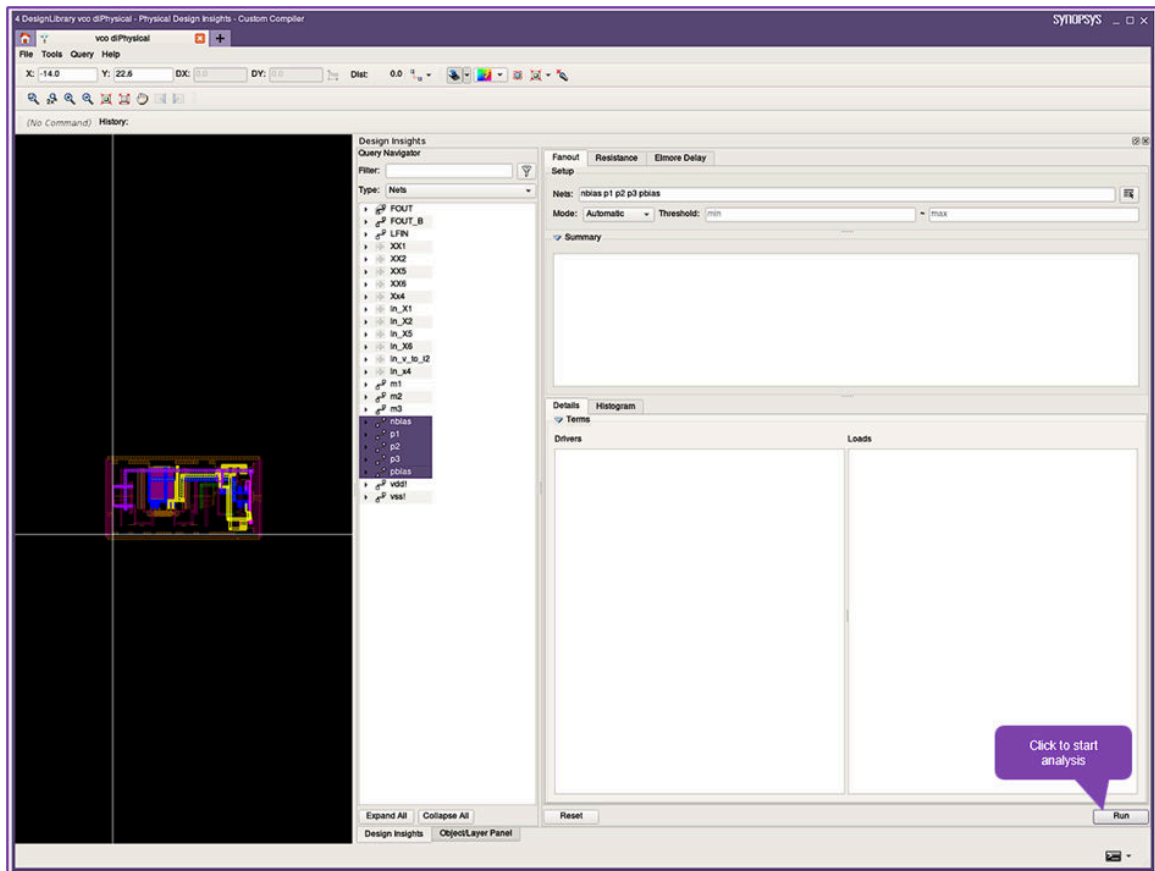
1. From **Query Navigator**, select the **Nets**, right-click and select **Analysis**.



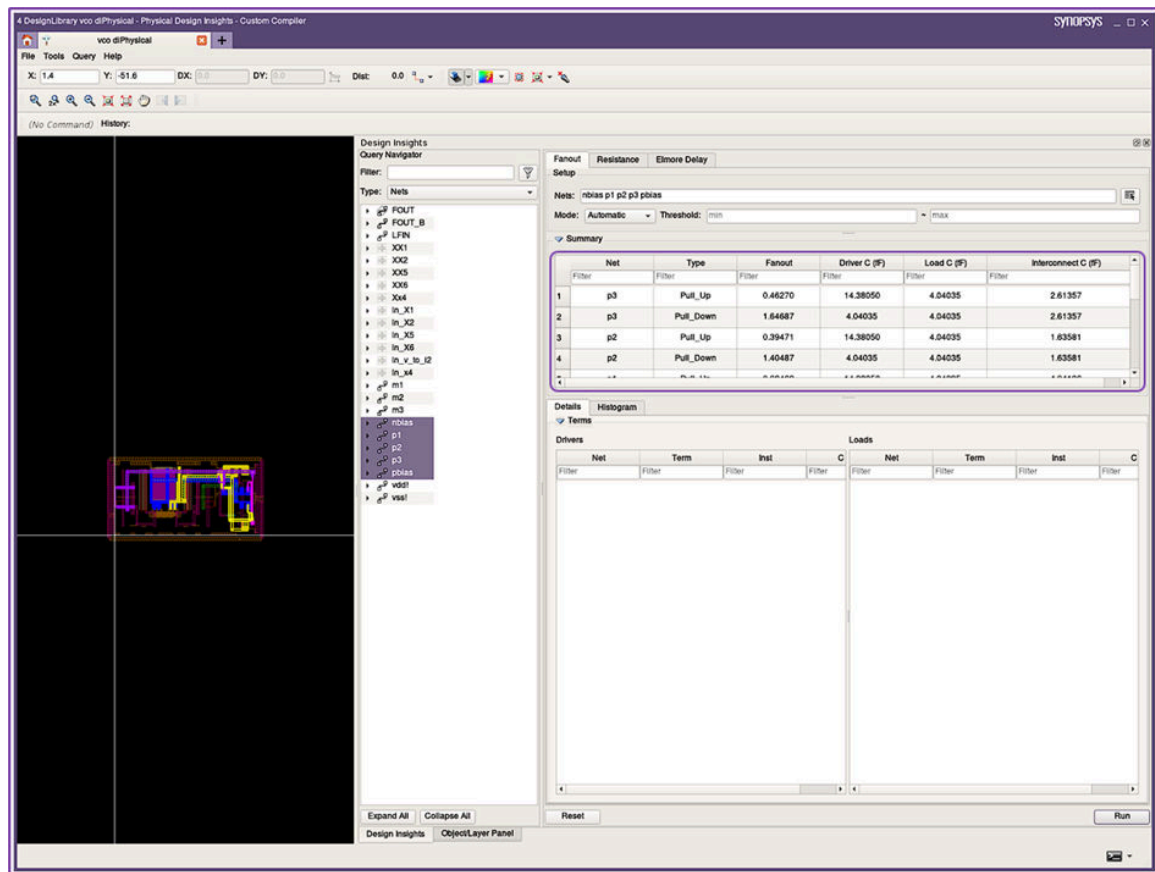
2. Select the required analysis in the context-sensitive menu and observe that the net names are filled in the Nets widget of the selected analysis tab. Here, Fanout analysis is selected.



3. Click **Run** to start the analysis.



4. After analysis is completed, the results are displayed as shown in the following image:

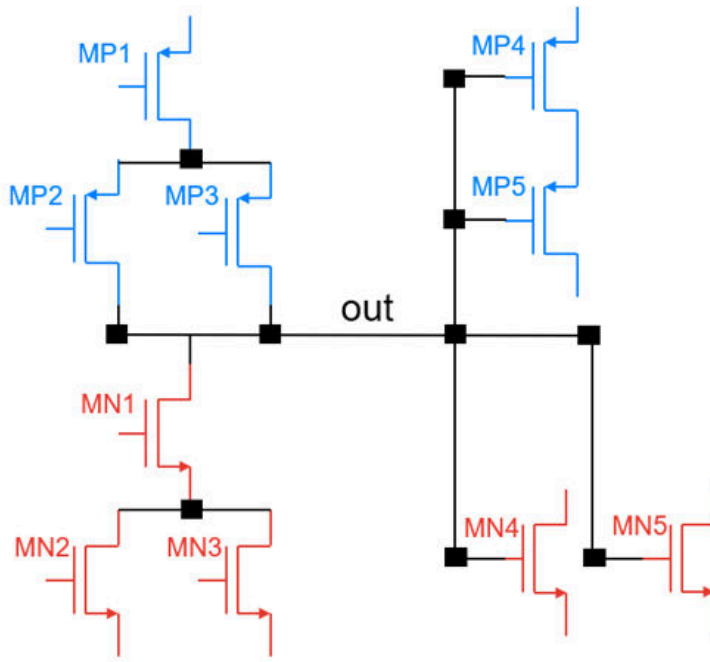


Fanout Analysis

Fanout is estimated based on the total gate size of drivers and loads.

$$Fanout = \frac{C_{total_loads} + C_{parasitic}}{C_{total_drivers}}$$

For example, consider the following circuit:



Here, for net out,

- C_{driver}
 - for pull-up path includes C_{P2} and C_{P3}
 - for pull-down path includes C_{N1}
- C_{load} includes C_{P4} , C_{P5} , C_{N4} and C_{N5}
- $C_{parasitic}$ is equal to C_{out}

Then, for the pull up path,

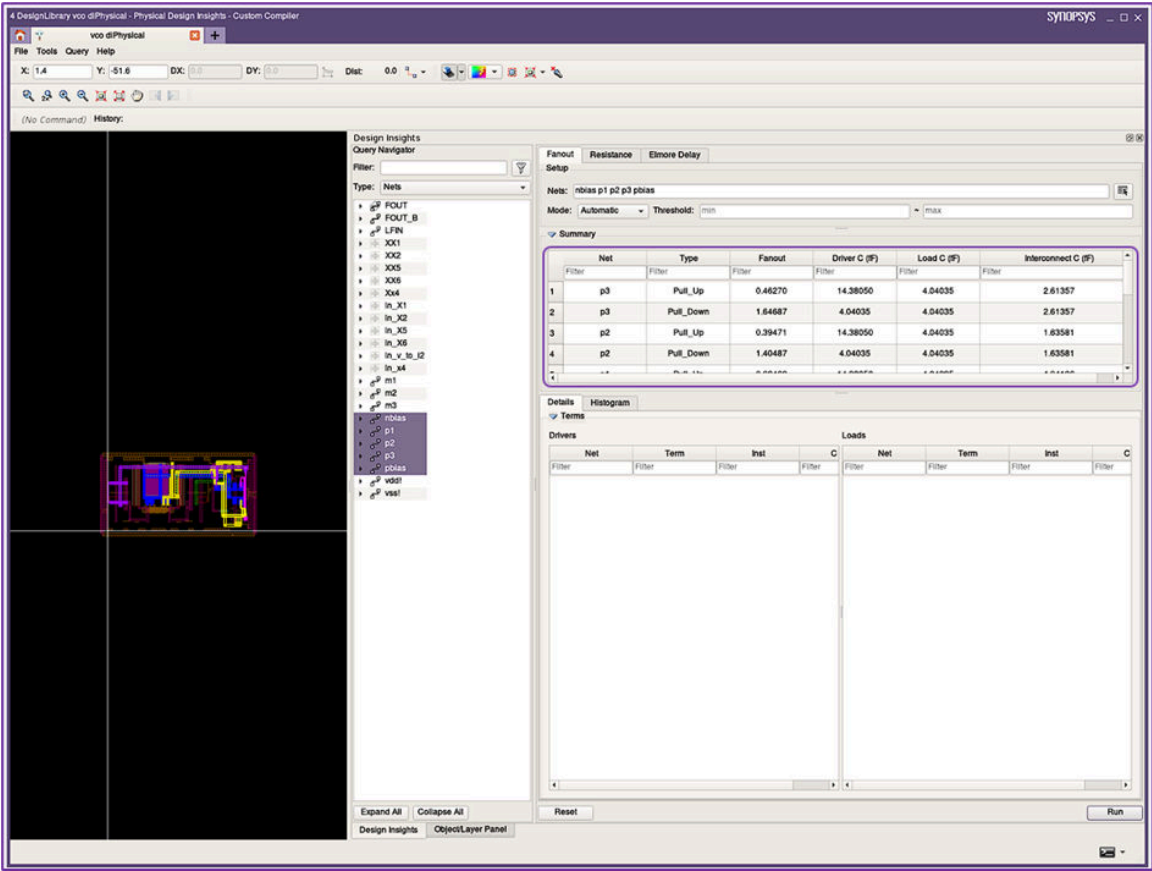
$$Fanout_{pull-up} = \frac{(C_{P4} + C_{P5} + C_{N4} + C_{N5}) + (C_{out})}{(C_{P2} + C_{P3})}$$

For the pull-down path,

$$Fanout_{pull-down} = \frac{(C_{P4} + C_{P5} + C_{N4} + C_{N5}) + (C_{out})}{(C_{N1})}$$

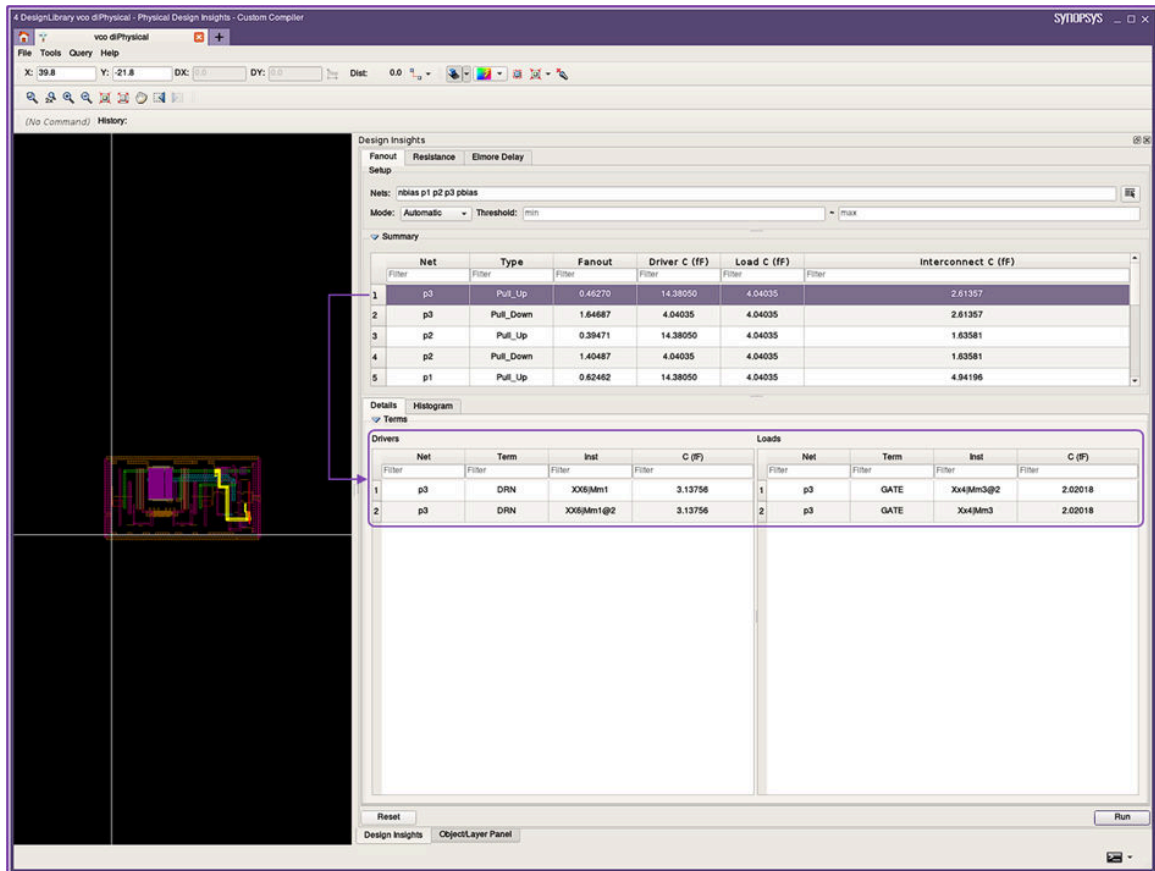
In the Design Insights GUI, the results are organized as:

- **Summary:** This table displays all the paths identified by the tool for the specified nets. For each path, the following information is displayed:
 - **Fanout**
 - **Driver C (fF)**
 - **Load C (fF)**
 - **Interconnect C (fF)**



Note:
Select an entry in the **Summary** table to highlight the net on the canvas.

- **Details:** This tab shows the terminals which are connected to the net selected in the **Summary** table. Each terminal's capacitance value is displayed to the user, and they are categorized into **Drivers** and **Loads**.



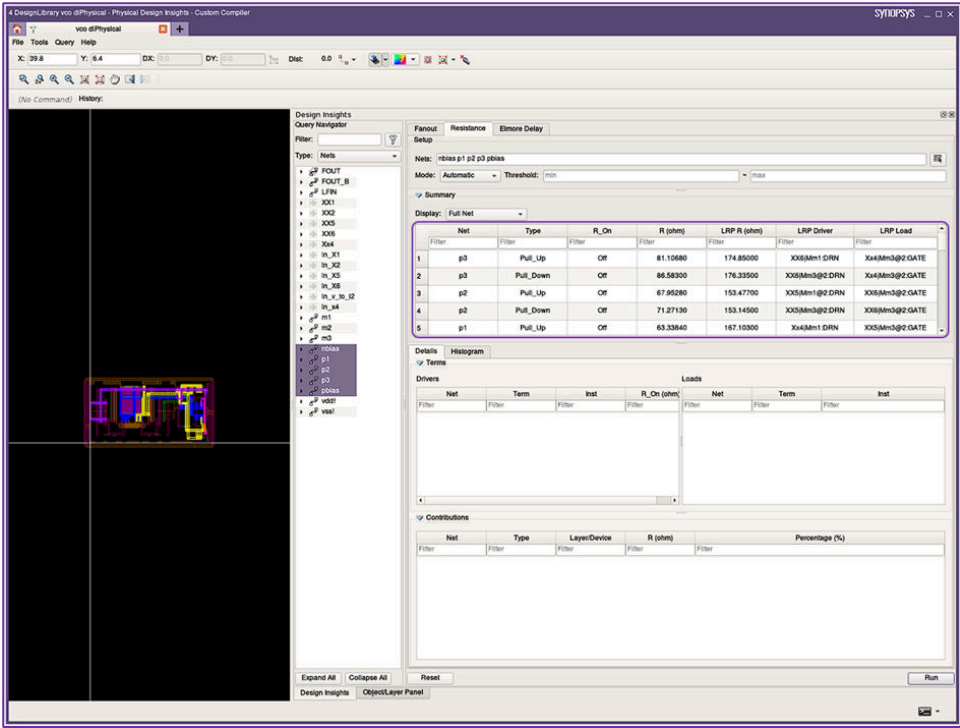
Understanding Resistance Analysis Results

Designers can run resistance analysis in Design Insights to get in-depth parasitic resistance information about each net. The following features are available within the tool to help with parasitic resistance debugging:

- Display Least Resistance Path (LRP)
- Check layer wise resistance contribution
- Calculate effective resistance for a path
- Perform point-to-point resistance checks for a specific net

After running resistance analysis, the **Summary** table shows the following information:

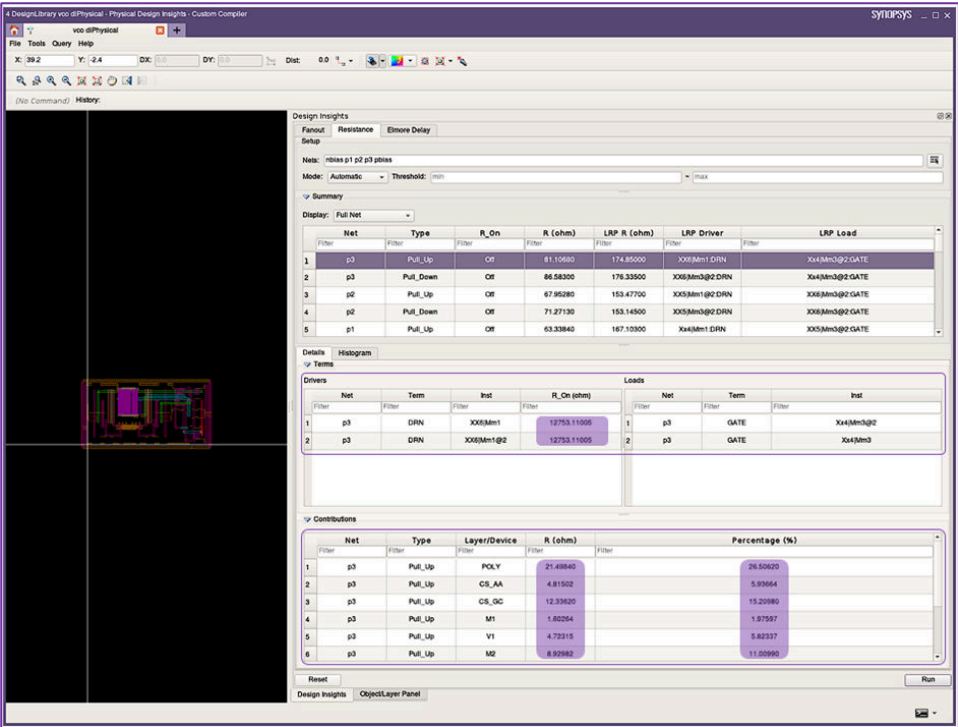
- **R (ohm)** - effective resistance of the path
- **LRP R (ohm)** - resistance of the least resistance path of the selected path
- **LRP Driver** - driver terminal name for the LRP
- **LRP Load** - load terminal name for the LRP



Check The Device ON Resistance (R_{on}) and Layer Wise Contribution

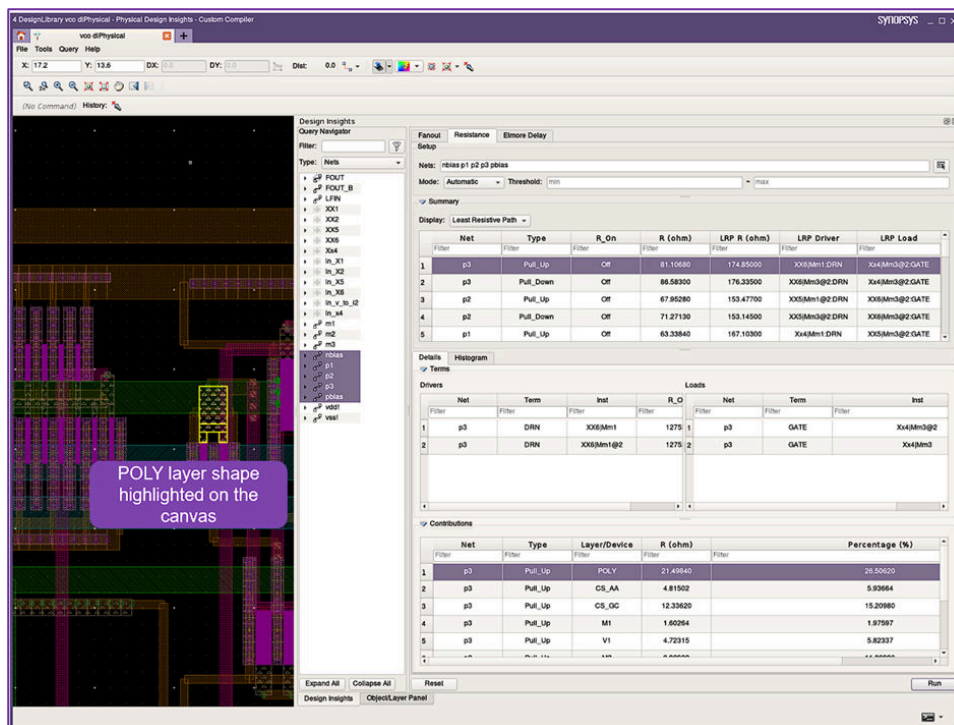
Select an entry in the **Summary** table to fill the tables within **Details** tab with appropriate information. There are three tables in this tab:

- **Drivers** - lists all the driver (drain/source) terminals of the selected path along with the R_{on} resistance of the devices.
- **Loads** - lists all the load (gate) terminals of the selected path.
- **Contributions** - provides the layer-wise resistance contribution of the selected path. This table shows the resistance contributed by a particular layer and also it's percentage contribution.



Note:
Click any entry in the **Contributions** table to highlight the corresponding shapes on the canvas.

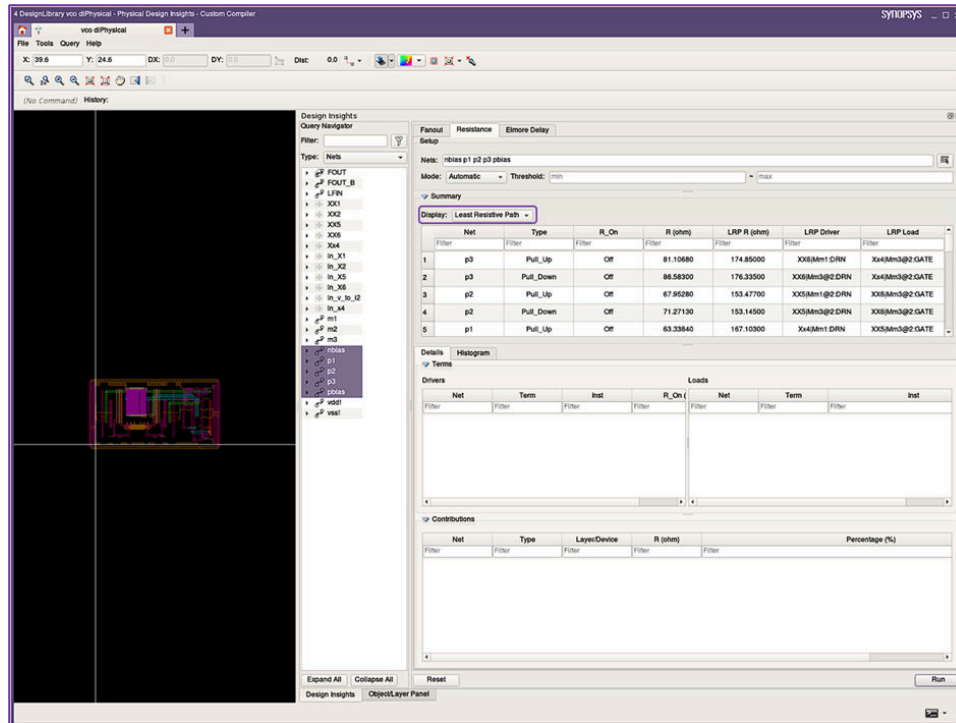
Chapter 1: Advanced Design Exploration With Design Insights Analyzing Parasitic



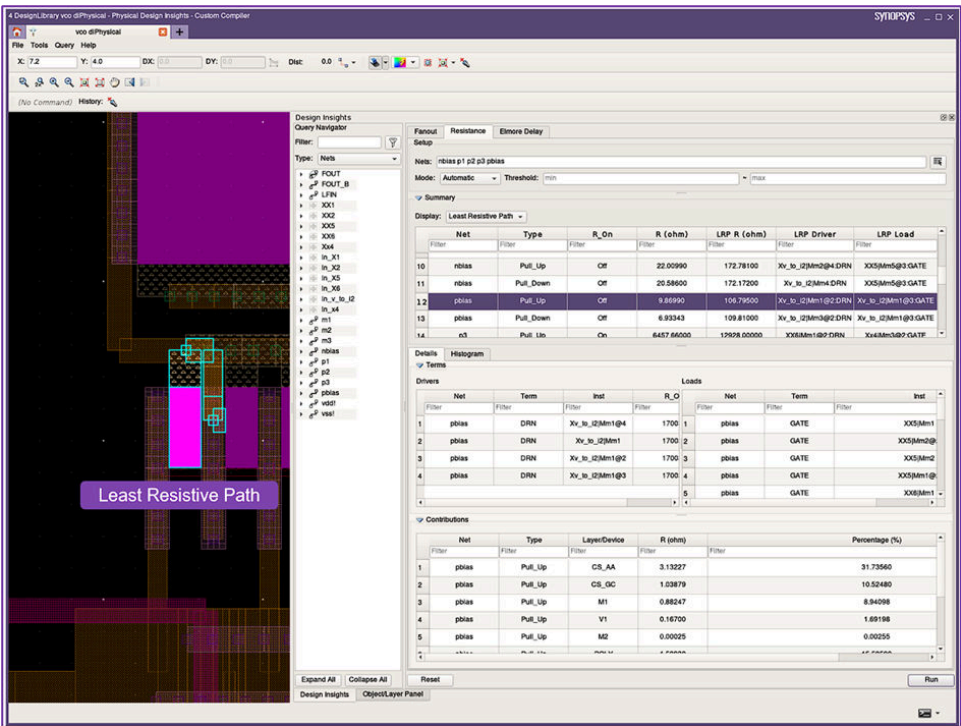
View the LRP Path on Canvas

To view the LRP path on canvas:

1. In the **Summary** section, set **Display:Least Resistive Path**.



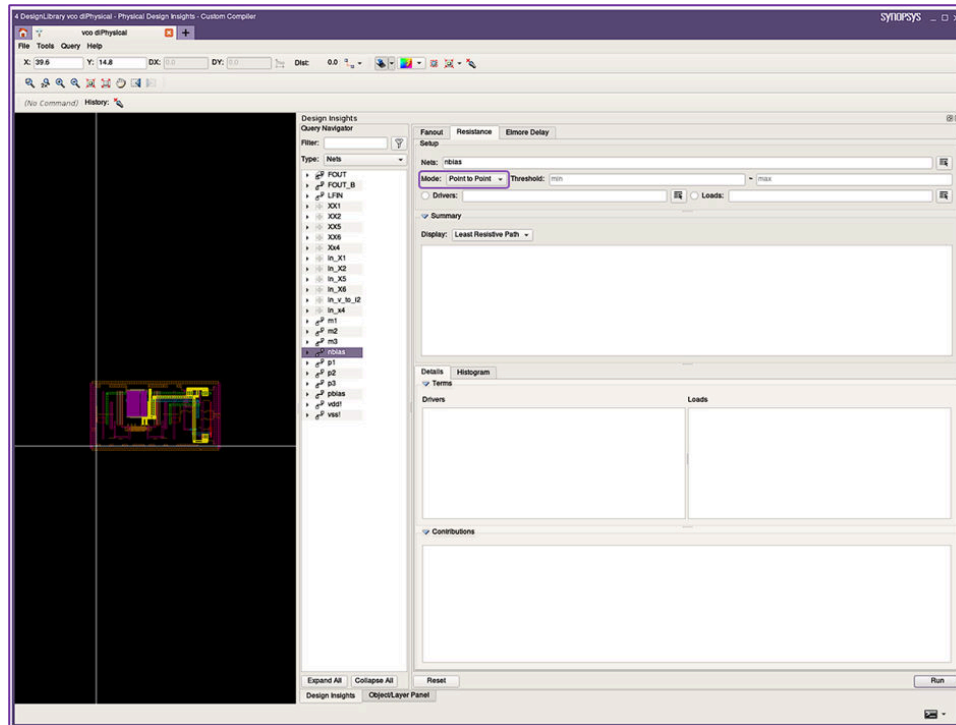
2. Select any entry in the **Summary** table to view the LRP path on canvas.



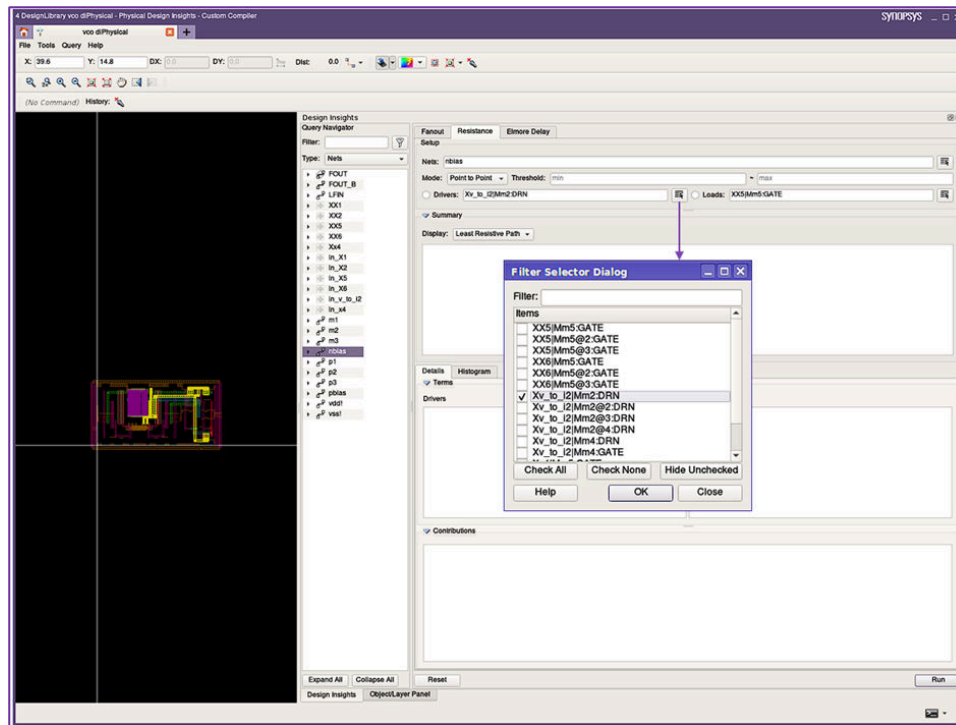
Perform Point-to-Point Resistance Checks

To run point-to-point resistance analysis:

1. Select a net for resistance analysis and set **Mode: Point-to-Point**.



2. Select the **Drivers** and **Loads** points using the **Filter Selector** dialog box.

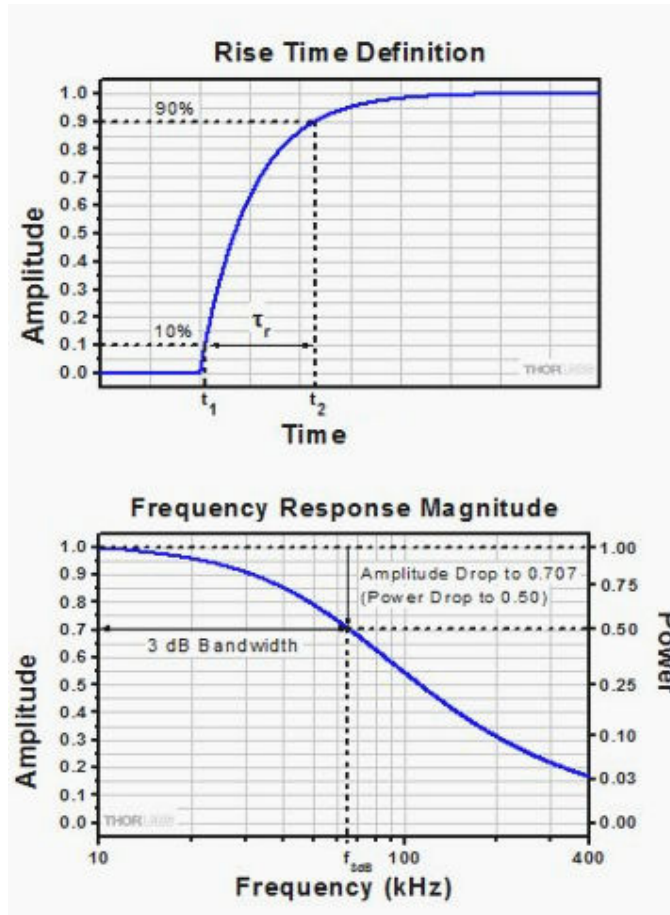


3. Click **Run** to start the analysis and the results can be checked as mentioned in the previous sections.

Understanding Elmore Delay Analysis Results

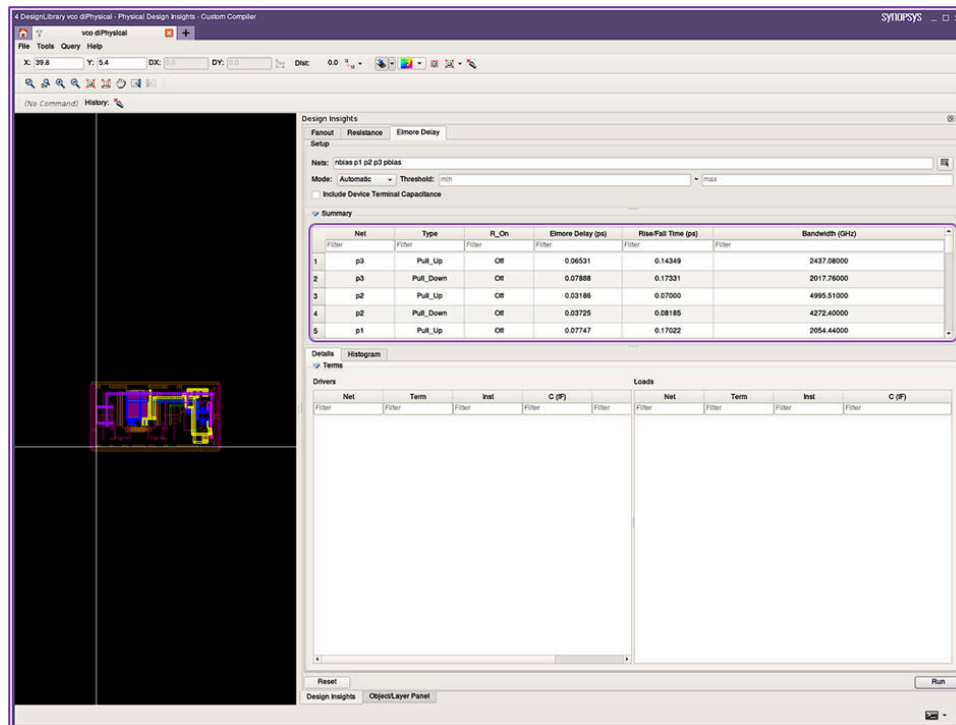
Elmore delay represents the RC time constant of a connection, based on which the temporal and frequency response information is provided.

- Time-domain response: With user-defined start and end amplitude rations, Design Insights provides rise and fall time from Elmore Delay.
- Frequency-domain response: Half-ratio power delivery frequency (-3 dB bandwidth).

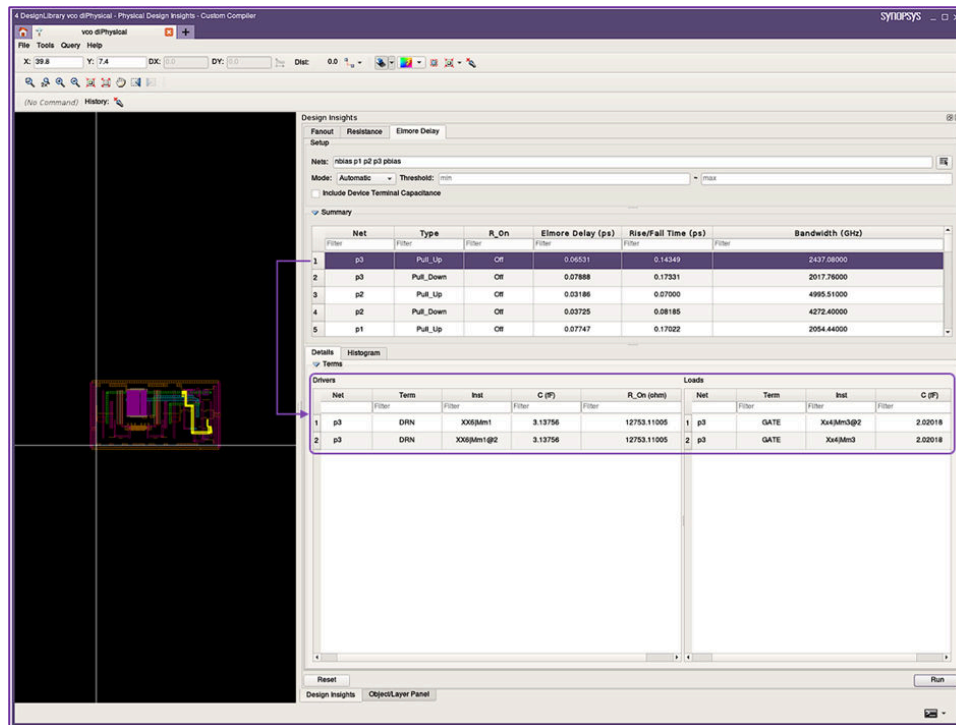


The results of Elmore Delay analysis are organized into,

- **Summary** - This table displays all the paths identified by the tool for the specified nets. For each path, the following information is displayed –
 - **Elmore Delay (ps)**
 - **Rise/Fall Time (ps)**
 - **Bandwidth (GHz)**



- **Details:** This tab shows the terminals which are connected to the net selected in the Summary table. Each terminal's capacitance value is displayed and they are categorized into Drivers and Loads. In addition, the R_{On} value for Drivers is displayed.



Note:

Delay analysis can be performed by enabling the **Include Device Terminal Capacitance** check box to incorporate device terminal capacitances.

To run Elmore Delay analysis in point-to-point mode, set **Mode**: Point-to-Point and follow the same steps as in the section Perform Point-to-Point Resistance checks.

Modes of Analysis in Design Insights

Design Insights analyses can be run in the following modes:

- Automatic - supported for Fanout, Resistance and Elmore Delay analysis
- Point-to-Point - supported for Fanout, Resistance and Elmore Delay analysis
- InstPin to Port - supported for Resistance analysis

Automatic Mode

- Pull-up
- Pull-down
- Port to Gate
- InstPin to Gate
- Port to Port
- InstPin to InstPin

ID	Path Type	Drivers	Loads
1	Pull-up	D1/D3 (Shorted)	G1/G2 (Shorted)
2	Pull-down	D2/D4 (Shorted)	
3	Port to Gate	A	
4		B	

ID	Path Type	Drivers	Loads
5	InstPin to Gate	N1	
6		N2	
7	InstPin to Port	D1/D3 (Shorted)	A
8			B
9		D2/D4 (Shorted)	A
10			B
11		N1	A
12			B
13		N2	A
14			B
15	Port to Port	A	B
16	InstPin to InstPin	D1/D3 (Shorted)	N1
17			N2
18		D2/D4 (Shorted)	N1
19			N2
20		N1	N2

Point to Point Mode

In this mode, analysis is run between two points of a net which are selected by the user.

InstPin to Port Mode

In this mode, analysis is run on a net by identifying all the paths from instance terminals to top-level ports in the design.

Histograms for Net Details and Analysis Result

When you select one or multiple nets in the summary table, a histogram can be displayed in the **Histogram** tab. You can select specific histogram of data to be displayed by changing the **Type** option. You can also adjust **Min**, **Max**, and **Color Level** to optimize histogram display.

Figure 2 shows the histogram for net comparison and Figure 3 shows the histogram for analysis results.

Figure 2 Histogram for Net Comparison

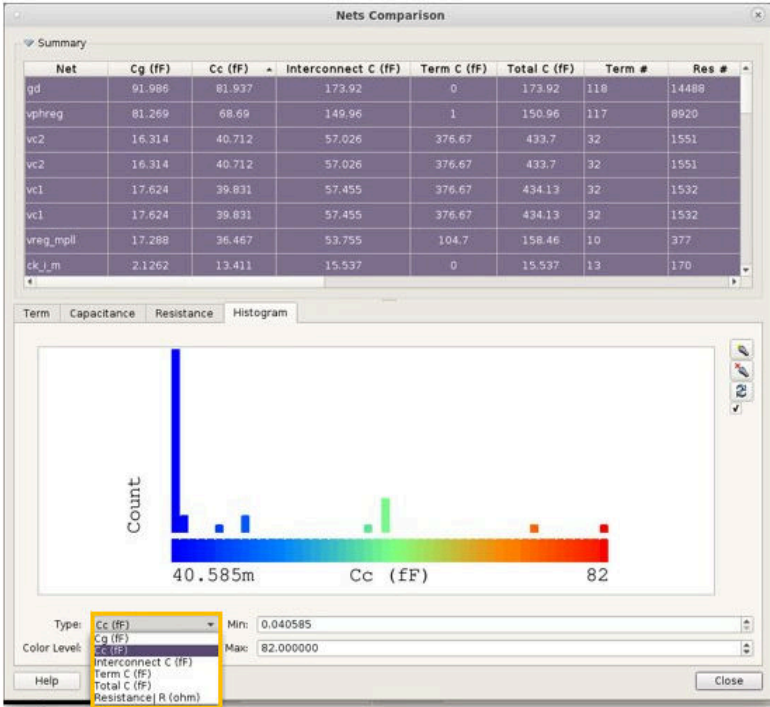
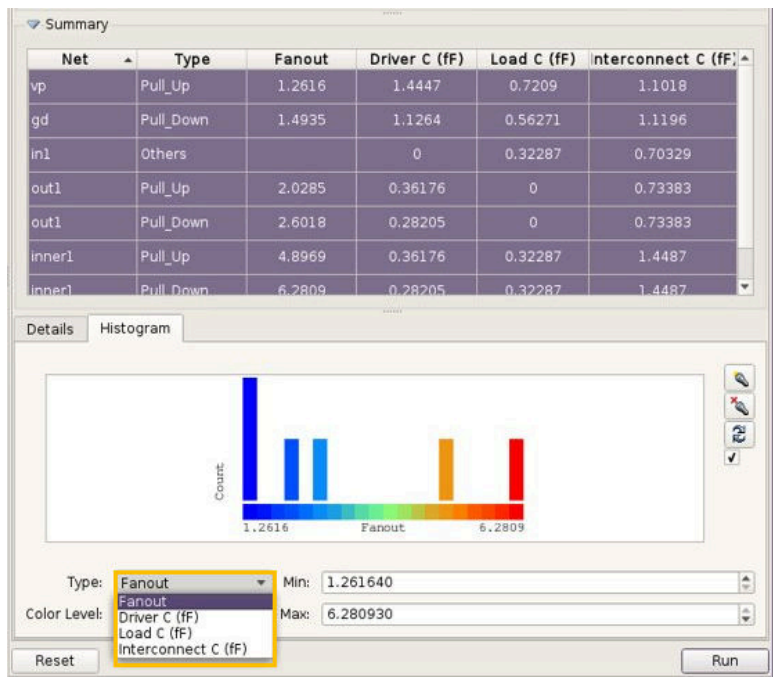
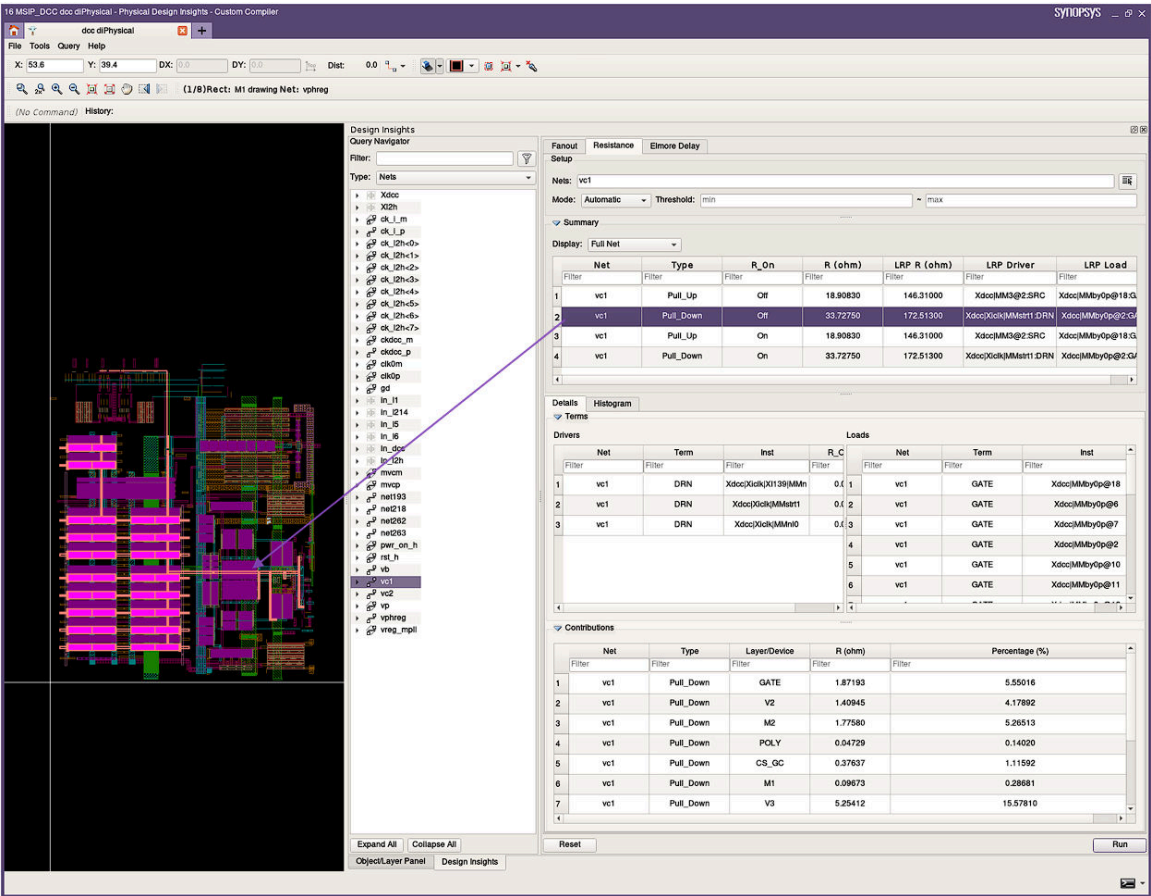


Figure 3 Histogram for Analysis Result

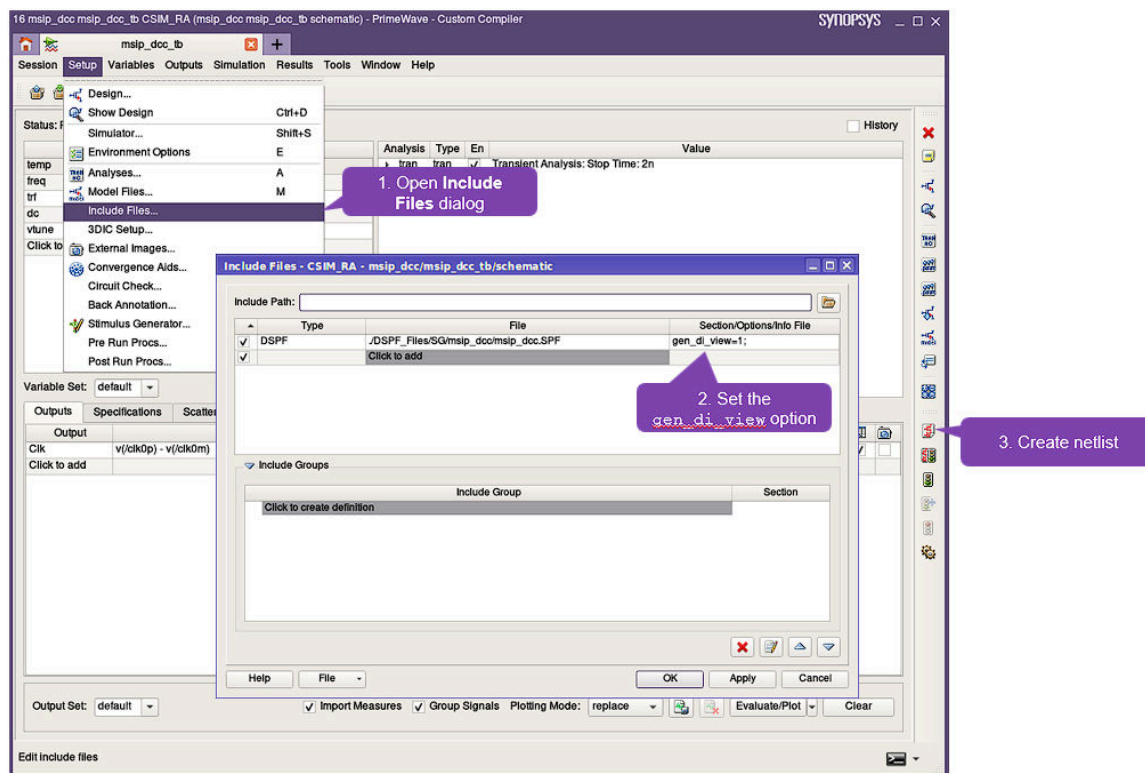


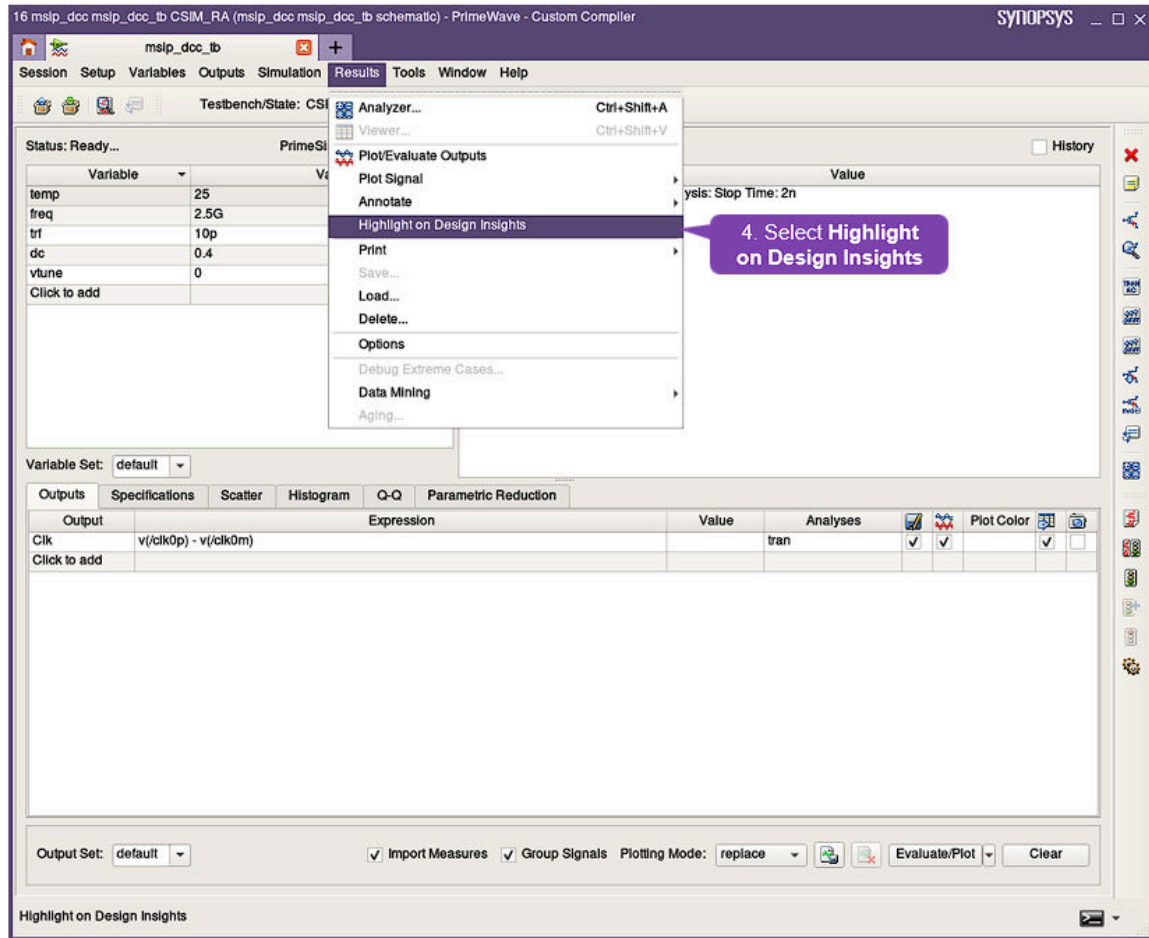
Cross Highlighting

Cross highlighting feature in Design Insights enables automatic highlighting in **Design Insights** view by selecting one or multiple target nets.

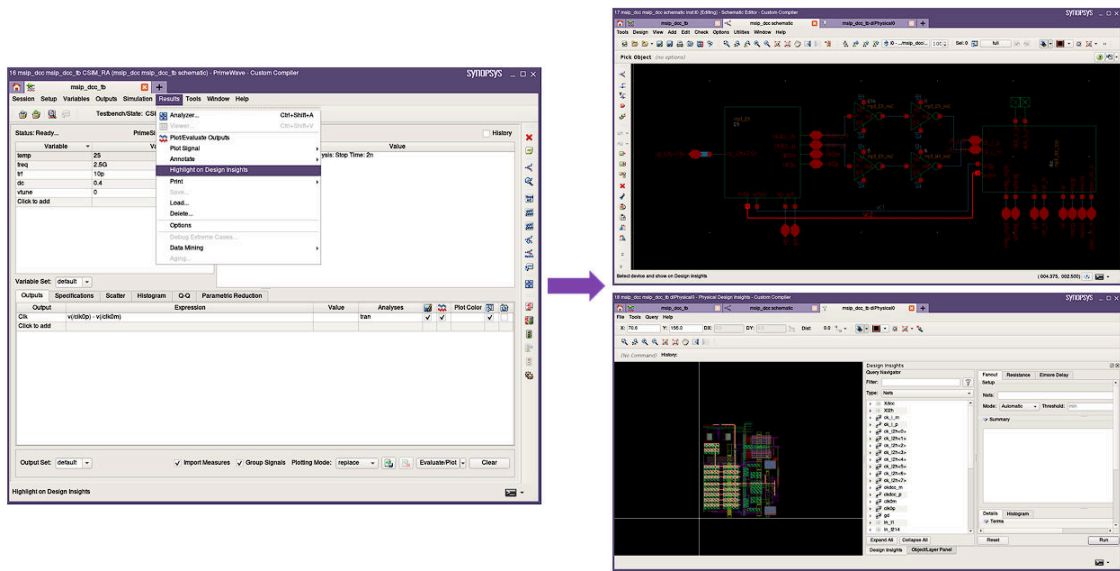


Starting with V-2023.12 release, the cross highlighting to Custom Compiler schematic is available through interfacing with PrimeWave Design Environment, the steps are described in following figures.





Chapter 1: Advanced Design Exploration With Design Insights
Cross Highlighting



Design Insights can generate hierarchical schematic based on DSPF input with logic cone back tracing capabilities to increase user debugging.

2

Design Insights Command Reference

This chapter details the Design Insights commands and preferences.

This chapter provides the list of Design Insights commands and preferences and detailed descriptions of the individual commands.

The commands in the Design Insights tool are grouped into the following categories:

- [di::closeResultsCellView](#)
- [di::createResultsCellView](#)
- [di::createParasiticView](#)
- [di::getCapacitors](#)
- [di::getCapAttr](#)
- [di::getCorners](#)
- [di::getElmoreDelayResult](#)
- [di::getElmoreDelayResultAttr](#)
- [di::getFanoutResult](#)
- [di::getFanoutResultAttr](#)
- [di::getInstAttr](#)
- [di::getInsts](#)
- [di::getLayers](#)
- [di::getModels](#)
- [di::getNetAttr](#)
- [di::getNets](#)
- [di::getNodeAttr](#)
- [di::getResAttr](#)

- [di::getResistanceContributionAttr](#)
- [di::getResistanceResult](#)
- [di::getResistanceResultAttr](#)
- [di::getResistors](#)
- [di::getTermAttr](#)
- [di::getTerms](#)
- [di::getViewType](#)
- [di::listAttrs](#)
- [di::loadResultsCellView](#)
- [di::runElmoreDelayAnalysis](#)
- [di::runFanoutAnalysis](#)
- [di::runResistanceAnalysis](#)
- [Preferences](#)

di::closeResultsCellView

Summary: Close the currently open design insights view.

Syntax: `di::closeResultsCellView`

Returns: `<none>`

Licenses: None.

di::createResultsCellView

Summary: Create design insights view by `rc_preprocessor` with `-path`.

Syntax: `di::createResultCellView[-selectNet <string> | -skipNet <string>]
-lcv<string> [-modelLibs <string>] [-modelSyntax <string>] [-overwrite
<bool>]-paths <string> -type <enum> [-useITFLayerName <bool>] [-vbias
<float>][-vbiasMappingFile <string>]`

Arguments:

`-netID` : Specifies the net id of the capacitors.

`-netName` : Specifies the net name of the capacitors.

-lcv: Target LCV name to create

-modelLibs: Specified the list of the lists to specify the corner, model libs and model sections.

-modelSyntax: Specifies the model syntax type. hspice, eldo, and spectre supported.

-overwrite: Specifies if overwrite parasitic view or not

-paths: Specifies the file paths rc_preprocessorread

-selectNet: Specifies the select net pattern

-skipNet: Specifies the skip net pattern

-type: Specifies the format the rc_preprocessorread. gpd and dspfformats supported. ({ dspf | gpd })

-useITFLayerName: Specify if the layer use ITF layer or not

-vbias: Specifies the vbias number

-vbiasMappingFile: Specified the list of the lists to specify the corner, model libs and model sections.

Returns:<xtJob>: Return the submitted job object

Licenses: None

```
di::createResultsCellView -lcv test2/test3/parasitic -paths p22.gpd -type  
gpd -useITFLayerName true -modelLibs  
{reference40_models.lib TT typical_25}{reference40_models.lib SS  
typical_125}{reference40_models.lib TT typical_M40}} -overwrite 1
```

di::createParasiticView

Summary: Create parasitic view by rc_preprocessor with -path.

Syntax: di::createParasiticView [-selectNet <string> | -skipNet <string>]
-lcv <string> [-modelLibs <string>] [-modelSyntax <string>] [-overwrite
<bool>] -paths <string> -type <enum> [-useITFLayerName <bool>] [-vbias
<float>] [-vbiasMappingFile <string>]

Arguments:

-lcv : Target LCV name to create

-modelLibs : Specified the list of the lists to specify the corner, model libs and model sections.

-modelSyntax : Specifies the model syntax type. hspice, eldo, and spectre supported.

`-overwrite` : Specifies if overwrite parasitic view or not

`-paths` : Specifies the file paths `rc_preprocessor read`

`-selectNet` : Specifies the select net pattern

`-skipNet` : Specifies the skip net pattern

`-type` : Specifies the format the `rc_preprocessor read`. `gpd` and `dspf` formats supported. `{ dspf | gpd }`

`-useITFLayerName` : Specify if the layer use ITF layer or not

`-vbias` : Specifies the vbias number

`-vbiasMappingFile` : Specified the list of the lists to specify the corner, model libs and model sections.

Returns: `<xtJob>` : Return the submitted job object

Licenses: None

di::getCapacitors

Summary: Report the capacitors' ids from the specified net id/name.

Syntax: `di::getCapacitors [-netID <int> | -netName <string>]`

Arguments:

`-netID` : Specifies the net id of the capacitors.

`-netName` : Specifies the net name of the capacitors.

Returns: `<string>` : Return list of the capacitors' ids.

Licenses: None

di::getCapAttr

Summary: Get capacitor data by specified capacitor id and attribute name.

Syntax: `di::getCapAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getCorners

Summary: Get corners information

Syntax: di::getCorners

Arguments: Returns: <string> : corners information

Licenses: None

di::getElmoreDelayResult

Summary: Get the Elmore Delay Result for current loaded parasitic view.

Syntax: di::getElmoreDelayResult [-net <string>] -type <enum>

Arguments:

-net : Specifies the target net name.

-type : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: <string> : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic
set net [di::getNets -name "FOUT"]
set job [di::runElmoreDelayAnalysis -mode automatic -nets $net -corners
"typical_M40 typical_125"]
xt::wait $job
set summary_ids [di::getElmoreDelayResult -type all]
foreach summary_id $summary_ids {
  set attrs [di::getElmoreDelayResultAttr -id $summary_id -attribute "net
elmore type bandwidth transitionTime drivers loads"]
  puts "net [lindex $attrs 0]"
  puts "elmore [lindex $attrs 1]"
  puts "type [lindex $attrs 2]"
  puts "bandwidth [lindex $attrs 3]"
  puts "transitionTime [lindex $attrs 4]"
  set drivers [lindex $attrs 5]
  set loads [lindex $attrs 6]
  puts "drivers info"
```

```
foreach driver $drivers {
  puts [di::getTermAttr -id $driver]
}
puts "loads info"
foreach load $loads {
  puts [di::getTermAttr -id $load]
}
}
```

di::getElmoreDelayResultAttr

Summary: Get the Elmore Delay Result for current loaded parasitic view.

Syntax: `di::getElmoreDelayResultAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getFanoutResult

Summary: Get the Fanout Result for current loaded parasitic view.

Syntax: `di::getFanoutResult [-net <string>] -type <enum>`

Arguments:

`-net` : Specifies the target net name.

`-type` : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: <string> : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic
set net [di::getNets -name "FOUT"]
set job [di::runFanoutAnalysis -mode automatic -nets $net]
xt::wait $job
```

```
set summary_ids [di::getFanoutResult -type all]
foreach summary_id $summary_ids {
  set attrs [di::getFanoutResultAttr -id $summary_id -attribute "net
fanout type driver_c interconnect_c load_c drivers loads"]
  puts "net [lindex $attrs 0]"
  puts "fanout [lindex $attrs 1]"
  puts "type [lindex $attrs 2]"
  puts "driver_c [lindex $attrs 3]"
  puts "interconnect_c [lindex $attrs 4]"
  puts "load_c [lindex $attrs 5]"
  set drivers [lindex $attrs 6]
  set loads [lindex $attrs 7]
  puts "drivers info"
  foreach driver $drivers {
    puts [di::getTermAttr -id $driver]
  }
  puts "loads info"
  foreach load $loads {
    puts [di::getTermAttr -id $load]
  }
}
```

di::getFanoutResultAttr

Summary: Get the Fanout Result Attribute data from current fanout result.

Syntax: di::getFanoutResultAttr [-attribute <string>] [-corners <string>]
-id <int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getInstAttr

Summary: Get instance data by specified instance id and attribute name.

Syntax: di::getInstAttr [-attribute <string>] [-corners <string>] -id
<int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getInsts

Summary: Report the list of the instances' ids from the netlist.

Category: Design Insights

Syntax: `di::getInsts [-name <string> | -netId <int> | -netName <string> | -termID <int> | -termName <string>]`

Arguments:

-name : Specifies the name of the instance.

-netId : Specifies net name of the instance.

-netName : Specifies net name of the instance.

-termID : Specifies term id of the net.

-termName : Specifies term name of the instance.

Returns: <string> : Return list of the instances' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set instances [di::getInsts -name XI1|MM0]
foreach instance $instances {
    set instance_attribute [di::getInstAttr -id $instance -attribute
    "name model type parameters"]
    puts "instance name [lindex $instance_attribute 0] information"
    puts "-----"
    puts "model [lindex $instance_attribute 1]"
    puts "type [lindex $instance_attribute 2]"
    puts "parameters [lindex $instance_attribute 3]"
}
di::closeResultsCellView
```

di::getLayers

Summary: Get layers information

Syntax: `di::getLayers [-layerName <string>]`

Arguments:

`-layerName` : specified layer name

Returns: `<string>` : layers information

Licenses: None

di::getModels

Summary: Get models information

Syntax: `di::getModels [-modelName <string>]`

Arguments:

`-modelName` : specified model name

Returns: `<string>` : models information

Licenses: None

di::getNetAttr

Summary: Get net data by specified net id and attribute name.

Syntax: `di::getNetAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getNets

Summary: Report the list of the nets' ids from the netlist.

Syntax: `di::getNets [-name <string> | -termID <int> | -termName <string>]`

Arguments:

`-name` : Specifies the name of the net

`-termID` : Specifies term id of the net.

`-termName` : Specifies term name of the net.

Returns: `<string>` : Return list of the nets' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set nets [di::getNets -name "ck*"]
foreach net $nets {
    set net_attribute [di::getNetAttr -id $net -attribute "name cg cc
interconnect_c term_c total_c r_count term_count max_r max_r_layer"]
    puts "Net name [lindex $net_attribute 0] information"
    puts "-----"
    puts "cg [lindex $net_attribute 1]"
    puts "cc [lindex $net_attribute 2]"
    puts "interconnect_c [lindex $net_attribute 3]"
    puts "term_c [lindex $net_attribute 4]"
    puts "total_c [lindex $net_attribute 5]"
    puts "r_count [lindex $net_attribute 6]"
    puts "term_count [lindex $net_attribute 7]"
    puts "max_r [lindex $net_attribute 8]"
    puts "max_r_layer [lindex $net_attribute 9]"
    puts "-----"
}
```

di::getNodeAttr

Summary: Get node data by specified node id and attribute name.

Syntax: `di::getNodeAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getResAttr

Summary: Get resistor data by specified resistor id and attribute name.

Syntax: di::getResAttr [-attribute <string>] [-corners <string>] -id <int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

```
di::loadParasiticDB -lcv di_test/di/parasitic
set resistances [di::getResistors -netName net193]
puts "dump res"
foreach resistance $resistances {
    # just get net node1 node2 position(llx lly urx ury) and value
    set resistance_attribute [di::getResAttr -id $resistance -attribute
"net node1 node2 llx lly urx ury value"]
    puts "net [lindex $resistance_attribute 0]"
    puts "node1 [lindex $resistance_attribute 1]"
    puts "node2 [lindex $resistance_attribute 2]"
    puts "position \{[lindex $resistance_attribute 3] , [lindex
$resistance_attribute 4], [lindex $resistance_attribute 5], [lindex
$resistance_attribute 6]\}"
    puts "value [lindex $resistance_attribute 7]"
}

puts "dump caps"

set capacitances [di::getCapacitors -netName net193]
foreach cap $capacitances {
    set cap_attribute [di::getCapAttr -id $cap -attribute "net1 node1
net2 node2 value"]
    puts "net1 [lindex $cap_attribute 0]"
    puts "node1 [lindex $cap_attribute 1]"
    puts "net2 [lindex $cap_attribute 2]"
    puts "node2 [lindex $cap_attribute 3]"
    puts "value [lindex $cap_attribute 4]"
}
```

```
}  
di::closeParasiticDB
```

di::getResistanceContributionAttr

Summary: Get the Effective Resistance Contribution Attribute data from opened effective resistance result.

Syntax: `di::getResistanceContributionAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getResistanceResult

Summary: Get the Effective Resistance Result for current loaded parasitic view.

Syntax: `di::getResistanceResult [-net <string>] -type <enum>`

Arguments:

`-net` : Specifies the target net name.

`-type` : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: `<string>` : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic  
set from [di::getTerms -name pfd|inv1|M1:GATE]  
set to [di::getTerms -name pfd|inv1|M3:GATE]  
set job [di::runResistanceAnalysis -mode p2p -drivers $from -loads $to]  
xt::wait $job  
set summary_ids [di::getResistanceResult -type all]  
foreach summary_id $summary_ids {  
  set attrs [di::getResistanceResultAttr -id $summary_id -attribute "net r  
  lrp_load lrp_driver lrp_r type drivers loads contributions"]
```

```
puts "net [lindex $attrs 0]"
puts "r [lindex $attrs 1]"
puts "lrp_load [lindex $attrs 2]"
puts "lrp_driver [lindex $attrs 3]"
puts "lrp_r [lindex $attrs 4]"
puts "type [lindex $attrs 5]"
set drivers [lindex $attrs 6]
set loads [lindex $attrs 7]
set contributions [lindex $attrs 8]
puts "drivers info"
foreach driver $drivers {
    puts [di::getTermAttr -id $driver]
}
puts "loads info"
foreach load $loads {
    puts [di::getTermAttr -id $load]
}
puts "contribution infos"
foreach contribution $contributions {
    puts [di::getResistanceContributionAttr -id $contribution]
}
}
```

di::getResistanceResultAttr

Summary: Get the Effective Resistance Result for current loaded parasitic view.

Syntax: di::getResistanceResultAttr [-attribute <string>] [-corners <string>] -id <int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getResistors

Summary: Report the resistors' ids from the specified net id/name.

Category: Design Insights

Syntax: di::getResistors [-netID <int> | -netName <string>]

Arguments:

-netID : Specifies the net id of the resistors.

-netName : Specifies the net name of the resistors.

Returns: <string> : Return list of the resistors' ids.

Licenses: None

di::getTermAttr

Summary: Get term data by specified term id and attribute name.

Syntax: `di::getTermAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getTerms

Summary: Report the list of the terms' ids from the netlist.

Syntax: `di::getTerms [-instID <int> | -instName <string> | -name <string> | -netID <int> | -netName <string>]`

Arguments:

-instID : Specifies instance id of the terms.

-instName : Specifies instance name of the terms.

-name : Specifies the name of the terms.

-netID : Specifies net id of the terms.

-netName : Specifies net name of the terms.

Returns: <string> : Return list of the terms' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set terms [di::getTerms -netName net193]
foreach term $terms {
    set term_attribute [di::getTermAttr -id $term -attribute "name nodes
net cap ron type"]
    puts "term name [lindex $term_attribute 0] information"
    puts "-----"
    puts "nodes [lindex $term_attribute 1]"
    puts "net [lindex $term_attribute 2]"
    puts "cap [lindex $term_attribute 3]"
    puts "ron [lindex $term_attribute 4]"
    puts "type [lindex $term_attribute 5]"
}
di::closeResultsCellView
```

di::getViewType

Summary: Get view type.

Syntax: di::getViewType

Returns: <string> : View type

Licenses: None

di::listAttrs

Summary: List all attributes for specified type of the di::get*Attr commands

Syntax: di::listAttrs -type <enum>

Arguments:

-type : List all attributes for specified type of the di::get*Attr commands. ({ cap | elmoreDelayResult | fanoutResult | inst | net | node | res | resistanceContributionResult | resistanceResult | term })

Returns: <string> : The list of the attribute name

Licenses: None

di::loadResultsCellView

Summary: Open a PA view.

Syntax: di::loadResultsCellView -lcv <dmCellView | dmCellViewName>

Arguments:

-lcv : Specify a PA cell view to be opened

Returns: <none>

Licenses: None

di::runElmoreDelayAnalysis

Summary: Get the Elmore delay result for currently loaded parasitic view.

Syntax: di::runElmoreDelayAnalysis [-corners <string>] [-drivers <string>]
[-includeDeviceTermCapacitance <bool>] [-loads <string>] -mode <enum>
[-nets <string>]

Arguments:

-corners : Specify the list of the corner name.

-drivers : Specifies the list of the driver's term id.

-includeDeviceTermCapacitance : Specify if the analysis have to include term cap or not.

-loads : Specifies the list of the load's term id.

-mode : Specifies mode of the command. ({ automatic | p2p })

-nets : Specifies list of the net id.

Returns: <xtJob> : Return the submitted job object

Licenses: None

di::runFanoutAnalysis

Summary: Get the fanout result for currently loaded parasitic view.

Syntax: di::runFanoutAnalysis [-corners <string>] [-drivers <string>]
[-loads <string>] -mode <enum> [-nets <string>]

Arguments:

-corners : Specify the list of the corner name.

-drivers : Specifies the list of the driver's term id.

-loads : Specifies the list of the load's term id.

`-mode` : Specifies mode of the command. ({ automatic | p2p }) `-nets` : Specifies list of the net id.

Returns: `<xtJob>` : Return the submitted job object

Licenses: None

di::runResistanceAnalysis

Summary: Get the effective resistance result for current loaded parasitic view.

Syntax: `di::runResistanceAnalysis [-corners <string>] [-drivers <string>] [-loads <string>] -mode <enum> [-nets <string>]`

Arguments:

`-corners` : Specify the list of the corner name.

`-drivers` : Specifies the list of the driver's term id.

`-loads` : Specifies the list of the load's term id.

`-mode` : Specifies mode of the command. ({ automatic | instP2p | p2p })

`-nets` : Specifies list of the net id.

Returns: `<xtJob>` : Return the submitted job object

Licenses: None

Preferences

The following table provides the preferences supported for Design Insights.

diCanvasClear	int	cellview	0	Clear	
diCanvasCrossSelectMode	bool	cellview	false	Cross select mode	true, false
diCanvasFitMode	bool	cellview	false	Fit mode	true, false
diCanvasShadowMode	bool	cellview	false	Shadow mode	true, false
diHighlightPacket	string	cellview		Highlight packet	

diPreprocessorOaOutputDir	string	cellview		OA output directory	
diPreprocessorRonMode	int	cellview	0	R_on Mode	
diPreprocessorSqlOutputDir	string	cellview		SQL output directory	
diPwdeHighlightToSchematic	bool	cellview	false	Highlight to schematic	true, false
diQueryNavigatorExpandAllWithConnectivity	bool	cellview	true	Expand all with connectivity	true, false
diShowNonPhysicalResistors	bool	cellview	false	Show non-physical resistors	true, false